Efficient Polynomial System Solving via Dixon Resultants: Applications to AO Primitives

Haohai Suo and Jiamin Cui

School of Cyber Science and Technology, Shandong University, China
haohai.suo@mail.sdu.edu.cn
cuijiamin@sdu.edu.cn

Abstract. Solving multivariate polynomial systems is a fundamental problem in cryptanalysis, with increasing relevance in algebraic attacks on arithmetization-oriented (AO) primitives. Current approaches primarily rely on Gröbner bases or the Sylvester resultant. However, Gröbner basis methods typically rely on FGLM for change of ordering, which applies only to zero-dimensional ideals, while the Sylvester resultant eliminates only one variable at a time, limiting its applicability to flexible multivariate elimination.

We revisit the Dixon resultant as an efficient and flexible tool for eliminating several variables simultaneously. We derive refined upper bounds on the Dixon matrix size via lattice-path counting and analyze the complexity under several determinant computation models, yielding explicit complexity estimates. For well-determined systems, the Dixon resultant is a viable alternative to Gröbner basis methods, with complementary strengths for under-determined system elimination and over-determined system solving.

We present an efficient open-source C implementation, `DRSolve`, with multiple determinant methods and a degree-aware submatrix selection strategy to mitigate the impact of extraneous factors. Experiments show that our implementation is competitive with the state-of-the-art Gröbner basis solvers Magma and msolve on randomly generated well-determined systems, with significant advantages in the low-variable/high-degree regime, while Magma and msolve remain preferable in the high-variable/low-degree regime.

Finally, we formulate three elimination strategies for polynomial systems arising from AO primitives: direct elimination, iterative elimination, and reduction-based hybrid elimination. We demonstrate these strategies on Poseidon, `Vision`, and `XHash12`, yielding complexity reductions in many cases.

Keywords: Dixon resultant, AO primitives, Poseidon, `Vision`, `XHash12`

1 Introduction

Solving multivariate polynomial systems over finite fields is a basic task in computational algebra. Three major viewpoints dominate the area: Gröbner basis methods [70,31,30], resultant-based elimination [26,58,50], and characteristic-set methods in the spirit of Wu's method [74]. In contemporary algebraic cryptanalysis, most attention is paid to Gröbner basis techniques [65,51], while resultant methods—especially the Dixon resultant as a compact multivariate elimination tool—remain much less explored in the algebraic cryptanalysis literature.

These problems are especially relevant for *arithmetization-oriented* (AO) primitives, whose round functions are deliberately designed to admit compact arithmetic representations in proof systems, MPC, and homomorphic settings. AO primitives differ substantially from classical binary-oriented designs. Instead of working over small binary fields, they are typically defined over large prime fields or extension fields, and their nonlinear layers are optimized for arithmetic circuits. Following this design philosophy, AO primitives are commonly organized into three families (Types I–III)[1]:

- *Type I (low degree):* MiMC [2], GMiMC [1], Poseidon [38], Poseidon2 [39], `Neptune` [41];
- *Type II (equivalence relations):* `Vision`/`Rescue` [4], `XHash12` [5], `Anemoi` [22], `Arion` [64], `Griffin` [35];
- *Type III (lookup tables):* `Reinforced Concrete` [36], `Monolith` [37], `Tip5` [71].

In all three settings, algebraic preimage and collision attacks against these primitives reduce to *constrained-input constrained-output* [17] (CICO) problems, which in turn amount to solving induced polynomial systems whose complexity dominates the security analysis.

Related works. Solving-based algebraic attacks have seen continuous refinement over the last several years. A major line of work [65,12,68,51,57,40,7] studies Gröbner basis attacks and, in particular, how to obtain better algebraic models or tighter upper bounds for their complexity on structured AO systems. In [51], the authors refined the complexity analysis by providing more precise estimates for the individual stages of the attack. However, in some cases the upper bounds obtained from such generic Gröbner basis analyses remain inaccurate. For example, the *Freelunch* approach [12] involves leveraging dedicated (weighted) monomial orderings for which a given polynomial system is already a Gröbner basis, allowing for better complexity bounds.

Another line of work brings classical elimination theory into cryptanalysis. Yang et al. [75] introduced iterative pairwise resultant attacks based on the Sylvester resultant, eliminating one variable at a time from two equations. Bariant et al. [11] further improved this paradigm by exploiting structure inside the polynomial system and performing degree reductions during elimination.

[1] See the STAP Zoo taxonomy: https://stap-zoo.com/zk/.

These are important developments, but they remain rooted in iterated two-polynomial resultants rather than *multipolynomial* resultants, so the elimination order is comparatively rigid.

Dixon resultant. The Dixon resultant, introduced by A.C. Dixon in 1909 [29], offers a different determinant-based elimination mechanism: it can eliminate several variables simultaneously from a polynomial system and thus serves as a compact multipolynomial resultant in our setting. Compared with the Macaulay resultant [58], the Dixon construction often leads to substantially smaller matrices while preserving the essential elimination information. Kapur et al. [50] also explained how to handle the non-square situation via maximal-rank submatrices, which makes the method viable beyond the classical idealized setting. Tang and Feng [72] used this construction to solve multivariate systems by keeping one variable as a parameter and solving the resulting eliminant, and Albrecht [3] implemented the same workflow against a block-cipher system; our direct method (Section 5.1) follows this workflow, our additions being the degree-aware submatrix selection of Section 4.1 and the fast polynomial-matrix determinant stage. Subsequently, numerous works [54,55,76,28,63,62,46,45] studied both the algebraic and algorithmic aspects of Dixon matrices.

For complexity analysis, the size of the Dixon matrix is the first key parameter. Kapur and Saxena [48] gave a Newton-polytope characterization for unmixed systems together with volume-based bounds for multi-homogeneous systems, and Tang and Feng [72] treated the degree-2 case. Mixed supports were studied by Chtcherba and Kapur [24] in the bivariate Dixon formulation; we generalize this to an arbitrary number of variables and obtain an explicit ordered mixed-total-degree formula, which is the form needed for the estimates in cryptographic applications. A second key parameter is the cost of polynomial-matrix determinants themselves, as expansion by minors [34], Bareiss elimination [43], Kronecker substitution [56], Hermite normal form [52], evaluation-interpolation [43], and sparse interpolation [44] can lead to substantially different complexity estimates, and we accordingly develop a stage-wise complexity model that compares all of them.

Our contributions This paper studies Dixon resultants as a method for *solving polynomial systems*, with AO primitives as a representative application domain. Our main contributions are as follows.

1. We derive refined bounds on Dixon matrix size for mixed systems via lattice-path counting. For uniform-degree systems, the bound specializes to the Fuss–Catalan numbers [42,6]; for general mixed systems, we obtain an explicit (Hessenberg) determinant formula, which is attained by the monomial support of generic instances.
2. We develop a stage-wise complexity model for Dixon elimination by comparing multiple determinant computation strategies, yielding a more faithful description of polynomial-matrix behavior.

3. We implement an efficient C library `DRSolve`[2] supporting multiple determinant routines and a degree-aware submatrix heuristic. Experiments validate the proposed Dixon-matrix bounds and show competitiveness with Magma and msolve on well-determined random systems, with advantages in the low-variable/high-degree/large-field regime, while Magma and msolve remain superior in the high-variable/low-degree/small-field regime.
4. We formulate three Dixon-based elimination strategies for AO-induced systems: direct, iterative, and hybrid reduction. We illustrate them on POSEIDON, `Vision`, and `XHash12`, respectively, providing concrete algebraic complexity estimates.

Outline. Section 2 reviews the Dixon construction and the determinant computation models used in the paper. Section 3 develops the refined Dixon-matrix bounds and the stepwise complexity analysis. Section 4 discusses implementation-oriented improvements. Section 5 presents the three application patterns on POSEIDON, `Vision`, and `XHash12`.

2 Preliminaries

Notation. We denote fields with $\mathbb{K}$ and their algebraic closure by $\overline{\mathbb{K}}$. Consider a system $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{a}][\boldsymbol{x}]$ of n equations in $n - 1 + k$ variables, where $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ are the eliminated variables and $\boldsymbol{a}$ are the retained parameters. We view each f_i as a polynomial in $\boldsymbol{x}$ with coefficients in $\mathbb{K}[\boldsymbol{a}]$. Any $f \in \mathbb{K}[\boldsymbol{a}][\boldsymbol{x}]$ can be written as

$$f = \sum_{\boldsymbol{u}} c_{\boldsymbol{u}} \boldsymbol{x}^{\boldsymbol{u}} = \sum_{\boldsymbol{u}} c_{\boldsymbol{u}} \prod_{i=1}^{n-1} x_i^{u_i},$$

where $c_{\boldsymbol{u}} \in \mathbb{K}[\boldsymbol{a}]$ and $\boldsymbol{u} = (u_1, \ldots, u_{n-1})$ is the exponent vector.

The total degree of f is denoted by $\deg(f)$, and the degree of f in the variable x_i is denoted by $\deg_{x_i}(f)$. For a polynomial system $\mathcal{G} = \{g_1, \ldots, g_r\} \subset \mathbb{K}[y_1, \ldots, y_s]$, $\langle \mathcal{G} \rangle$ denotes the ideal generated by $\mathcal{G}$.

2.1 Resultants

Resultants are classical tools in elimination theory used to determine whether a polynomial system has a common solution. We use the projection-operators viewpoint of [49].

Definition 1 (Resultant [49]). *Let $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ be the variables to eliminate and $\boldsymbol{a} = (a_1, \ldots, a_k)$ the retained parameters. For a system $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{a}][\boldsymbol{x}]$, let $I = \langle \mathcal{F} \rangle \subset \mathbb{K}[\boldsymbol{a}, \boldsymbol{x}]$ and $J := I \cap \mathbb{K}[\boldsymbol{a}]$. Polynomials in J are called* projection operators *and vanish on the parameter values for which $\mathcal{F}$ has a common solution over $\overline{\mathbb{K}}$. If J is principal, its generator (up to scalar) is the* resultant *of $\mathcal{F}$.*

[2] https://github.com/drsolve/drsolve

The Sylvester resultant [26] for two univariate polynomials of degrees d_1, d_2 produces a matrix of size $d_1 + d_2$; the Bézout–Cayley resultant [61,26] reduces this to $\max(d_1, d_2)$. The Macaulay resultant [58] generalizes this to several multivariate polynomials but can produce very large matrices. The Dixon construction [29] is a multivariate extension of the Bézout-style approach: it preserves the compactness of the Bézout–Cayley matrix while extending it from two to n polynomials, often yielding much smaller matrices than Macaulay.

Remark 1. When $k > 1$, J is generally not principal, and determinant-based methods compute a specific non-zero element $D \in J$. The affine determinantal construction (including the Dixon resultant) is defined as the determinant of a maximal-rank submatrix of the Dixon matrix $\mathcal{M}$ of Section 2.2 and vanishes on the projection of the affine solution set, but may include extraneous factors not essential for the projection. In this paper, we refer to this determinant D as the resultant in a slightly informal sense, rather than projection operators.

2.2 Dixon Resultant

The Dixon resultant method [29,50] eliminates several variables simultaneously via a determinant. The procedure has four steps [50,45].

Step 1: Compute the Dixon polynomial. Introduce $n-1$ auxiliary variables $\overline{x}_1, \ldots, \overline{x}_{n-1}$ and define the $n \times n$ cancellation matrix

$$C(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \begin{bmatrix} f_1(x_1, x_2, \ldots, x_{n-1}) & \cdots & f_n(x_1, x_2, \ldots, x_{n-1}) \\ f_1(\overline{x}_1, x_2, \ldots, x_{n-1}) & \cdots & f_n(\overline{x}_1, x_2, \ldots, x_{n-1}) \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, x_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, x_{n-1}) \\ \vdots & \ddots & \vdots \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) \end{bmatrix}. \tag{1}$$

The Dixon polynomial is

$$\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \frac{\det(C(\boldsymbol{x}, \overline{\boldsymbol{x}}))}{\prod_{i=1}^{n-1}(x_i - \overline{x}_i)}. \tag{2}$$

Instead of performing an explicit symbolic division, we use the refined construction from [46]

$$\mathrm{Row}(\widehat{C}_1) = \mathrm{Row}(C_1), \qquad \mathrm{Row}(\widehat{C}_{i+1}) = \frac{\mathrm{Row}(C_{i+1}) - \mathrm{Row}(C_i)}{x_i - \overline{x}_i}, \tag{3}$$

for $i = 1, \ldots, n-1$. Each numerator is divisible by $x_i - \overline{x}_i$ in $\mathbb{K}[\boldsymbol{x}, \overline{\boldsymbol{x}}, \boldsymbol{a}]$, so the quotient remains polynomial; the Dixon polynomial is $\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \det(\widehat{C})$. This row-wise difference-quotient construction is used throughout the paper.

Step 2: Construct the Dixon matrix. The Dixon polynomial admits a bilinear form $\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = X\mathcal{M}\overline{X}^T$, where X and $\overline{X}$ list monomials in $\boldsymbol{x}$ and $\overline{\boldsymbol{x}}$, respectively. The coefficient matrix $\mathcal{M}$ is the **Dixon matrix**: its rows are indexed by the $\boldsymbol{x}$-monomials and its columns by the $\overline{\boldsymbol{x}}$-monomials occurring in Δ, so it is in general rectangular and rank-deficient. Alternatively, $\mathcal{M}$ can be constructed directly via the recursive block method of Zhao and Fu [76], particularly relevant when few variables are eliminated and degrees are high [62].

Step 3: Extract a maximal-rank submatrix. In practice, the Dixon matrix $\mathcal{M}$ is often rectangular or rank-deficient. To obtain a valid elimination polynomial, we extract a maximal-rank square submatrix $\mathcal{M}'$, of size $\operatorname{rank}\mathcal{M}$, by selecting linearly independent rows and columns from $\mathcal{M}$. Following [46,45], this selection is typically performed efficiently after evaluating $\mathcal{M}$ at a generic point of $\mathbb{K}$.

Remark 2. The validity of using $\det(\mathcal{M}')$ as a non-zero projection operator relies on a specific rank condition on the singular Dixon matrix, as established by KSY condition [50]. While there are theoretical cases where this condition might not hold, such instances are extremely rare in practice. In our experiments, it was satisfied for every instance reported in this paper. If a degenerate case does arise (i.e., even maximal-rank submatrices yield trivial outputs), it can be easily resolved by fixing a single variable x_i to a random value and recomputing.

Step 4: Compute the Dixon resultant. Once $\mathcal{M}'$ is obtained, $\det(\mathcal{M}')$ is the Dixon resultant in $\mathbb{K}[\boldsymbol{a}]$. Under the standard Dixon hypotheses, it vanishes whenever the original system has a common solution in the eliminated variables and is a multiple of the principal generator of I_{elim} whenever that ideal is principal. In general, $\det(\mathcal{M}')$ may contain extraneous factors.

Size of the Dixon matrix. The size of the Dixon matrix is a key parameter in the complexity analysis. A classical variable-wise upper bound [50,62] is

$$B_{var} = (n-1)! \prod_{i=1}^{n-1} m_i, \tag{4}$$

where $m_i = \max_{1 \le j \le n} \deg_{x_i}(f_j)$ is the maximum degree in x_i over all polynomials in $\mathcal{F}$. For multi-homogeneous systems, Kapur and Saxena [48] proposed the volume-based bound

$$B_{MH} = \left\lfloor \frac{(dn)^{n-1}}{n!} \right\rfloor. \tag{5}$$

Section 3 replaces these estimates by exact generic formulas for the total-degree setting relevant here.

Influence of extraneous factors. The determinantal output may contain factors unrelated to the affine solution set.

Definition 2 (Extraneous factor). *Let* $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{x}, \boldsymbol{a}]$ *and* $I = \langle \mathcal{F} \rangle$. *An irreducible* $e \in \mathbb{K}[\boldsymbol{a}]$ *is an* ***extraneous factor*** *if* e *divides the determinantal elimination polynomial but* $e \notin I \cap \mathbb{K}[\boldsymbol{a}]$.

In the absence of such factors and of excess multiplicities, the degree of the eliminant coincides with $d_{\mathcal{I}}$ when $k = 1$, and its total degree is still bounded by $d_{\mathcal{I}}$ when $k \geq 2$; in both cases it is bounded by the Bézout bound:

Theorem 1 (Bézout bound). *Let* $\mathcal{I} = \langle f_1, \ldots, f_m \rangle \subset \mathbb{K}[x_1, \ldots, x_m]$ *be zero-dimensional, where* $\deg(f_i) = d_i$, *then*

$$d_{\mathcal{I}} \leq \prod_{i=1}^{m} d_i.$$

In the presence of extraneous factors, the degree of the eliminant may exceed the Bézout bound of the underlying system, both inflating computational costs and introducing spurious components unrelated to the actual solution set.

2.3 Polynomial Matrix Determinant Computation

We summarize the determinant complexity models used in this paper; full details are in Appendix I.

Univariate determinants. For $P(t) \in \mathbb{K}[t]^{m \times m}$, the Hermite normal form (HNF) method of [52] has deterministic cost $\mathcal{T}^{\mathrm{HNF}} = \widetilde{O}(m^{\omega} \lceil s \rceil)$, where s is the average column degree.

Multivariate determinants. For $P(\boldsymbol{y}) \in \mathbb{K}[y_1, \ldots, y_t]^{m \times m}$, no single method is universally best: different algorithms can lead to substantially different complexity estimates depending on the matrix size, the number of variables, and the sparsity structure of the entries. We therefore consider several alternatives. Besides direct symbolic expansion by minors [43,34] and fraction-free Bareiss elimination [10], one may reduce to the univariate case via Kronecker substitution [56] and apply HNF, apply evaluation-interpolation on a tensor grid [43], or use sparse interpolation [44]. The corresponding time and space costs are summarized in Table 1.

In the table notation, s denotes the matrix size, K is the induced univariate degree bound after Kronecker substitution (chosen to avoid overflow in the encoded determinant); $B := \max_i b_i$ is the largest variable-wise degree bound; $N := \prod_{i=1}^{t}(b_i + 1)$ is the tensor-grid size used by interpolation; S bounds the number of nonzero terms of $\det(P)$; U bounds the term count of intermediate minors; L is the cost of one determinant probe; and $\mu \leq \binom{t+d}{d}$ bounds the

Table 1: Determinant computation models used in this paper. All complexities are in field operations over $\mathbb{F}_q$.

Method	Time	Space
Expansion [34,43]	$\widetilde{O}(s2^s\mu U)$	$\widetilde{O}(\binom{s}{s/2}U)$
Bareiss [10,34]	$\widetilde{O}(s^3U^2)$	$\widetilde{O}(s^2U)$
Kronecker [56]	$\widetilde{O}(s^\omega K)$	$\widetilde{O}(sK)$
Interpolation [43]	$\widetilde{O}(N(L+tB))$	$\widetilde{O}(N)$
Sparse [44][3]	$\widetilde{O}(LS+S\log q)$	$\widetilde{O}(S)$

dense monomial count of an entry of total degree at most d in t variables. All complexities are reported in field operations over $\mathbb{F}_q$. Detailed explanations and derivations are given in Appendix I.

2.4 Security of AO Primitives

The sponge construction [16] builds hash functions from permutations. The relevant task for algebraic attacks is the constrained-input constrained-output [17] (CICO) problem.

Definition 3 (CICO problem). *For a permutation $F\colon \mathbb{F}_q^t \to \mathbb{F}_q^t$, the CICO-$k$ problem is to find $x \in \mathbb{F}_q^{t-k} \times \{0\}^k$ and $y \in \mathbb{F}_q^{t-k} \times \{0\}^k$ with $F(x) = y$.*

Brute force requires $\sim q^k$ permutation calls; substantially faster algebraic solutions may translate into preimage or collision attacks in the sponge setting.

3 Improved Complexity Estimates for Dixon Resultant

Prior work on Dixon resultant bounds [48] focused on multi-homogeneous and unmixed systems. We consider general mixed systems with varying degrees, bound the size of the Dixon matrix via lattice path counting, and obtain bounds that are attained for generic systems, yielding refined complexity estimates.

Throughout this section, $D(\mathcal{F})$ denotes the support size of the Dixon matrix $\mathcal{M}$ associated with $\mathcal{F}$, that is, the maximum of the numbers of distinct $\boldsymbol{x}$-monomials and $\overline{\boldsymbol{x}}$-monomials occurring in the Dixon polynomial. We denote a system of n polynomials with total-degree bounds $d_1, d_2, \ldots, d_n$ as a $(d_1, \ldots, d_n)$-system. The special case $d_1 = \cdots = d_n = d$ is referred to as an $(n; d)$-system, following [48].

[3] The sparse model also requires $\mathrm{char}(\mathbb{F}_q) \geq D$ (Huang [44, Theorem 1]); it therefore applies to large-prime fields but not to extension fields of the form $\mathbb{F}_{2^\ell}$.

3.1 A Refined Bound on the Size of the Dixon Matrix

We bound the Dixon matrix size for general $(d_1, \ldots, d_n)$-systems via a lattice path count. Background is given in Appendix A.

Theorem 2 (Lattice Path Bound). *For a $(d_1, d_2, \ldots, d_n)$-system $\mathcal{F}$ with $d_1 \le d_2 \le \cdots \le d_n$, let*

$$a_i := \sum_{k=1}^{i} (d_{n+1-k} - 1), \qquad i = 1, 2, \ldots, n-1.$$

Then $D(\mathcal{F})$ is bounded above by the number of lattice paths from $(0,0)$ to $(n-1, a_{n-1})$ such that for every $i = 1, 2, \ldots, n-1$, the height y_i of the i-th horizontal step is at most a_i. Moreover, if $\mathrm{char}(\mathbb{K}) = 0$ or $\mathrm{char}(\mathbb{K}) > \sum_{i=2}^{n}(d_i - 1)$, this bound is attained for generic polynomial systems.

Remark 3. Note that the minimum degree d_1 does not affect the size of the Dixon matrix. In the refined construction (3), the first row of $\widehat{C}$ still contains the original polynomials f_j, while the $\bar{\boldsymbol{x}}$-support of $\det \widehat{C}$ is determined solely by the difference-quotient rows $i = 2, \ldots, n$ (Lemma 4). The column corresponding to f_1 contributes only a factor of $\bar{\boldsymbol{x}}$-degree zero.

This lattice path counting problem admits precise closed-form solutions through classical results in enumerative combinatorics.

Lemma 1. *Let $a = (a_1, a_2, \ldots, a_{n-1})$ be an integer sequence with $0 \le a_1 \le a_2 \le \cdots \le a_{n-1}$. Consider lattice paths from $(0,0)$ to $(n-1, a_{n-1})$.*

(1) The total number of lattice paths without any restriction equals

$$\binom{(n-1) + a_{n-1}}{n-1}.$$

(2) If $a_i = i(d-1)$ for some constant d and all $i = 1, \ldots, n-1$, then the number of lattice paths such that the height y_i of the i-th horizontal step is at most a_i equals the Fuss–Catalan number with parameters (n, d)

$$C_n^{(d)} := \frac{1}{(d-1)n+1} \binom{dn}{n}.$$

(3) The number of lattice paths such that for every $i = 1, 2, \ldots, n-1$ the height y_i of the i-th horizontal step is at most a_i is given by the determinant

$$\det_{1 \le i,j \le n-1} \left(\binom{a_i + 1}{j - i + 1} \right).$$

Proof. Parts (1) and (2) are classical results in enumerative combinatorics; see [42,6]. Part (3) is a special case of Theorem 10.7.1 in [19] with $b_j = 0$ for all j.

By combining Theorem 2 and Lemma 1, we immediately obtain the following bounds.

Corollary 1. *For $(d_1, d_2, \ldots, d_n)$-system $\mathcal{F}$ with $d_1 \le d_2 \le \cdots \le d_n$, the following three bounds of Dixon matrix size hold:*

(1) ***Unrestricted Bound:***

$$D(\mathcal{F}) \le \binom{\sum_{i=2}^{n}(d_i - 1) + n - 1}{n-1} = \binom{\sum_{i=2}^{n} d_i}{n-1}. \tag{6}$$

This bound is obtained by counting all lattice paths from $(0, 0)$ to $(n - 1, \sum_{i=2}^{n}(d_i - 1))$ without boundary constraints.

(2) ***Fuss-Catalan Bound (for $(n; d)$-systems):*** *Assume $d_1 = \cdots = d_n = d$.*

$$D(\mathcal{F}) \le C_n^{(d)} = \frac{1}{(d-1)n+1}\binom{dn}{n}. \tag{7}$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(2).

(3) ***Determinant Bound:***

$$D(\mathcal{F}) \le \det_{1 \le i,j \le n-1}\left(\binom{\sum_{k=1}^{i}(d_{n+1-k} - 1) + 1}{j - i + 1}\right). \tag{8}$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(3).

Remark 4. The matrix in (8) is Hessenberg matrix, hence its determinant can be computed in $\mathcal{O}(n^2)$ using specialized algorithms [23] or [60, Theorem 1.9].

Outline of proof of Theorem 2. We introduce the Iterated-Simplex Polynomial as a bridge connecting the Dixon matrix size to lattice path counting. Lemma 2 gives a polyhedral description of its support via inequalities $\alpha_1 + \cdots + \alpha_k \le \sum_{i=1}^{k}(d_{n-i+1} - 1)$. Using this description, Lemma 4 shows the Dixon matrix size is bounded by the number of monomials of $I_{d_2-1,\ldots,d_n-1}$, with equality for generic systems. Lemma 5 then bijects these lattice points with lattice paths below a piecewise linear boundary.

Definition 4 (Iterated-Simplex Polynomial). *Let $n \ge 1$ be a positive integer and $(d_1, d_2, \ldots, d_n)$ be a sequence of positive integers. The Iterated-Simplex polynomial is defined as*

$$I_{d_1,d_2,\ldots,d_n}(x_1, \ldots, x_n) := \prod_{k=1}^{n}\left(\sum_{\alpha_1+\cdots+\alpha_k \le d_k} x_1^{\alpha_1} \cdots x_k^{\alpha_k}\right), \tag{9}$$

where the k-th factor represents the complete sum of monomials in the first k variables with total degree at most d_k.

Now we characterize the support of the Iterated-Simplex polynomial in terms of linear inequalities.

Lemma 2. *The support of the Iterated-Simplex Polynomial $I_{d_1,d_2,\ldots,d_n}$ is given by*

$$\mathrm{Supp}(I_{d_1,d_2,\ldots,d_n}) = \Big\{\boldsymbol{\alpha} \in \mathbb{Z}^n_{\geq 0} \;:\; \sum_{l=n-j+1}^{n} \alpha_l \leq \sum_{i=1}^{j} d_{n-i+1} \quad \text{for } j = 1, 2, \ldots, n\Big\}. \tag{10}$$

The proof is deferred to Appendix B, since it requires a careful argument for both inclusions in the support characterization.

To relate different orderings of degrees, we establish a monotonicity property showing that the sorted sequence has maximal support.

Lemma 3. *Let $(d_1, d_2, \ldots, d_n)$ be a sequence with $d_1 \leq d_2 \leq \cdots \leq d_n$, and let $(d'_1, d'_2, \ldots, d'_n)$ be any permutation of this sequence. Then*

$$\mathrm{Supp}(I_{d'_1,d'_2,\ldots,d'_n}) \subseteq \mathrm{Supp}(I_{d_1,d_2,\ldots,d_n}).$$

Proof. By Lemma 2, the support is determined by the inequalities $\alpha_{n-k+1}+\cdots+\alpha_n \leq \sum_{j=1}^{k} d_{n-j+1}$. For any permutation $(d'_1, \ldots, d'_n)$, the sum $\sum_{j=1}^{k} d'_{n-j+1}$ is bounded above by the sum of the k largest elements of the multiset $\{d_1, \ldots, d_n\}$, which is precisely $\sum_{j=1}^{k} d_{n-j+1}$ under the sorted ordering $d_1 \leq \cdots \leq d_n$. Hence the constraints defining $\mathrm{Supp}(I_{d'_1,\ldots,d'_n})$ are no weaker than those for $\mathrm{Supp}(I_{d_1,\ldots,d_n})$, implying the inclusion.

Connection to the Dixon Matrix Size. We now connect the Dixon matrix size to the Iterated-Simplex Polynomial.

Lemma 4. *Let $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[x_1, \ldots, x_{n-1}]$ be a system of polynomials with total degrees at most $d_1, d_2, \ldots, d_n$ respectively, where $d_1 \leq d_2 \leq \cdots \leq d_n$. Then $D(\mathcal{F})$ is bounded above by the number N^* of monomials in the Iterated-Simplex Polynomial $I_{d_2-1,d_3-1,\ldots,d_n-1}$, and this bound is attained for generic polynomial systems provided* $\mathrm{char}(\mathbb{K}) = 0$ *or* $\mathrm{char}(\mathbb{K}) > \sum_{i=2}^{n}(d_i - 1)$.

Proof. After the cancellation matrix (1) is constructed, the Dixon polynomial admits the bilinear form $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = X\mathcal{M}\overline{X}^T$ of Section 2, so the number of $\bar{\boldsymbol{x}}$-monomials and the number of $\boldsymbol{x}$-monomials are the row and column dimensions of $\mathcal{M}$. We therefore count the monomials of Δ in $\bar{\boldsymbol{x}}$ by treating the original variables as generic constants $x_i = c_i$ for $1 \leq i < n$. The cancellation matrix becomes:

$$C'(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \begin{bmatrix} f_1(c_1, c_2, \ldots, c_{n-1}) & \cdots & f_n(c_1, c_2, \ldots, c_{n-1}) \\ f_1(\overline{x}_1, c_2, \ldots, c_{n-1}) & \cdots & f_n(\overline{x}_1, c_2, \ldots, c_{n-1}) \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, c_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, c_{n-1}) \\ \vdots & \ddots & \vdots \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) \end{bmatrix}.$$

After applying the refined construction of the cancellation matrix from the Dixon polynomial definition, the leading determinantal expansion term is the product $\prod_{i=1}^{n} \widehat{C}_{i,i}$. The factor $\widehat{C}_{1,1}$ has $\bar{\boldsymbol{x}}$-degree 0 and contributes no $\bar{\boldsymbol{x}}$-monomial; the remaining $n-1$ factors realize the Iterated-Simplex Polynomial $I_{d_2-1,d_3-1,\ldots,d_n-1}$ with degrees in sorted order $d_2-1 \le d_3-1 \le \cdots \le d_n-1$. For any other permutation σ, the product $\prod_i \widehat{C}_{i,\sigma(i)}$ has its $\bar{\boldsymbol{x}}$-degree in row i bounded by $d_{\sigma(i)}-1$, so its $\bar{\boldsymbol{x}}$-support satisfies the inequalities of Lemma 2 with the permuted sequence $(d_{\sigma(2)}-1,\ldots,d_{\sigma(n)}-1)$. By Lemma 3 this support is contained in that of the sorted main-diagonal product, so the main-diagonal term carries the maximal monomial set. Therefore, the number of $\bar{\boldsymbol{x}}$-monomials of Δ is bounded above by the number of monomials in $I_{d_2-1,d_3-1,\ldots,d_n-1}$. The same bound holds for the number of $\boldsymbol{x}$-monomials of Δ by the dual argument: specializing $\bar{x}_i = c_i$ (instead of $x_i = c_i$) and repeating the construction yields the identical Iterated-Simplex bound on the $\boldsymbol{x}$-support of Δ, since the cancellation construction is symmetric in the two variable sets up to an overall sign. Both row and column dimensions of $\mathcal{M}$ are therefore bounded above by the iterated-simplex monomial count, which is the claimed bound.

To establish that the bound is achieved for generic systems, we specialize the system to a nested form $\mathcal{F}^* = \{f_1,\ldots,f_n\}$ where $f_1 = 1$ and for $i = 2,\ldots,n$:

$$f_i = \left(\sum_{k=1}^{i-1} x_k\right)^{d_i}.$$

Note that the iterated-simplex bound $I_{d_2-1,\ldots,d_n-1}$ does not depend on d_1, so taking f_1 to be a constant is consistent with the coefficient space of $(d_1,\ldots,d_n)$-systems used in the genericity argument below. Following the refined construction of the cancellation matrix $\widehat{C}$ (3), the rows are defined via recursive difference quotients. For $j < i$, the polynomial f_j depends only on $x_1,\ldots,x_{j-1}$, so it does not contain x_{i-1}; hence $C_{i,j} = C_{i-1,j}$ and the numerator of the difference quotient in row i vanishes identically, giving $\widehat{C}_{i,j} = 0$ for $j < i$. Consequently, $\widehat{C}$ is upper-triangular:

$$\widehat{C} = \begin{bmatrix} 1 & f_2 & f_3 & \cdots & f_n \\ 0 & M_{2,2} & M_{2,3} & \cdots & M_{2,n} \\ 0 & 0 & M_{3,3} & \cdots & M_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & M_{n,n} \end{bmatrix}$$

where the diagonal entries $M_{i,i}$ $(i = 2,\ldots,n)$ are given by:

$$M_{i,i} = \frac{(\bar{X}_{i-2}+\bar{x}_{i-1})^{d_i} - (\bar{X}_{i-2}+x_{i-1})^{d_i}}{x_{i-1}-\bar{x}_{i-1}} = -\sum_{k=0}^{d_i-1} (\bar{X}_{i-2}+x_{i-1})^k (\bar{X}_{i-2}+\bar{x}_{i-1})^{d_i-1-k}$$

with $\bar{X}_{i-2} = \sum_{j=1}^{i-2} \bar{x}_j$. The overall sign is irrelevant for the support analysis below.

Each $M_{i,i}$ is (a constant multiple of) a complete homogeneous symmetric form in the two arguments $\bar{X}_{i-2} + x_{i-1}$ and $\bar{X}_{i-2} + \bar{x}_{i-1}$. After specializing $x_{i-1} = c_{i-1}$ for a generic constant c_{i-1}, every term in the resulting expansion is a non-negative multinomial coefficient times a monomial in $\bar{x}_1, \ldots, \bar{x}_{i-1}$, so no coefficient cancellation occurs. Thus, the support of $M_{i,i}$ with respect to $\bar{x}_1, \ldots, \bar{x}_{i-1}$ is exactly a simplex of degree $d_i - 1$. The Dixon polynomial is obtained as $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \det(\widehat{C}) = \prod_{i=2}^{n} M_{i,i}$. The support of Δ is precisely characterized by the Iterated-Simplex Polynomial $I_{d_2-1,\ldots,d_n-1}$.

Attainment for generic systems follows from a Zariski-density argument. For $\mathcal{F}^*$ the row and column dimensions of $\mathcal{M}$ both equal the iterated-simplex monomial count N^*, which by the first part of the proof is the generic maximum. The condition that every iterated-simplex monomial appears in Δ with a nonzero coefficient is a finite conjunction of nonvanishing polynomial conditions on the coefficients of $\mathcal{F}$, all satisfied at $\mathcal{F}^*$; it therefore defines a non-empty, hence dense, Zariski-open subset of the irreducible coefficient space, on which the row and column dimensions of $\mathcal{M}$ equal N^*. Hence $D(\mathcal{F}) = N^*$ is attained generically. All multinomial coefficients arising in the relevant expansions have upper index at most $\sum_{i=2}^{n}(d_i - 1)$, so the stated characteristic assumption prevents their vanishing modulo $\mathrm{char}(\mathbb{K})$.

Remark 5. Throughout the matrix-size discussion, "generic" is understood in the usual Zariski-open sense of Definition 5, and $D(\mathcal{F})$ denotes the row/column dimension of $\mathcal{M}$, i.e. the size of its monomial support. This is an upper bound for, and in general strictly larger than, $\operatorname{rank}\mathcal{M}$: exhibiting all row and column monomials does not exhibit a non-vanishing minor of size N^*. Support containment, and hence the upper bound, holds over any field; only attainment uses the hypothesis on the characteristic. Over finite fields the formulas should be read as upper-bound templates after specialization rather than as exact equalities for every structured instance.

From Polyhedral Constraints to Lattice Paths We now establish a bijection between lattice points satisfying the polyhedral constraints and lattice paths below a general boundary.

Lemma 5. *Let $a_1 \le a_2 \le \cdots \le a_n$ be nonnegative integers. The number of lattice points $(\alpha_1, \ldots, \alpha_n)$ satisfying*

$$\alpha_1 + \cdots + \alpha_k \le a_k, \quad k = 1, \ldots, n,$$

is equal to the number of lattice paths from $(0, 0)$ to (n, a_n) such that for every $i = 1, 2, \ldots, n$ the height y_i of the i-th horizontal step is at most a_i.

Proof. There is a natural bijection between the two sets. Given $(\alpha_1, \ldots, \alpha_n)$ satisfying $\alpha_1 + \cdots + \alpha_k \le a_k$ for all $k = 1, \ldots, n$, set $\alpha_{n+1} := a_n - (\alpha_1 + \cdots + \alpha_n) \ge 0$ and define a lattice path by

$$U^{\alpha_1}, R, U^{\alpha_2}, R, \ldots, U^{\alpha_n}, R, U^{\alpha_{n+1}},$$

where R denotes a rightward step and U^m denotes m upward steps. The trailing $U^{\alpha_{n+1}}$ block pads the path so that it ends at (n, a_n). This is a path from $(0, 0)$ to (n, a_n) with exactly n horizontal steps. After the k-th horizontal step (for $k = 1, \ldots, n$), the height equals $\alpha_1 + \cdots + \alpha_k$, which is at most a_k. The height constraint is imposed only at the horizontal steps, i.e. on the partial sums $\alpha_1 + \cdots + \alpha_k$ with $k \leq n$; the trailing block of upward steps is unconstrained.

Conversely, any such path uniquely determines $(\alpha_1, \ldots, \alpha_n)$ by recording the number of upward steps immediately before each horizontal step. The height condition implies $\alpha_1 + \cdots + \alpha_k \leq a_k$ for all $k = 1, \ldots, n$.

We now combine the three lemmas above into a proof of Theorem 2. By Lemma 4 the Dixon matrix size is bounded by the monomial count of $I_{d_2-1, d_3-1, \ldots, d_n-1}$ (in $n-1$ variables and with $n-1$ subscripts), with equality for generic systems. By Lemma 2 applied with n replaced by $n-1$ and degree sequence $(d_2 - 1, \ldots, d_n - 1)$, this monomial count equals the number of lattice points $(\alpha_1, \ldots, \alpha_{n-1}) \in \mathbb{Z}_{\geq 0}^{n-1}$ satisfying

$$\sum_{\ell=n-j}^{n-1} \alpha_\ell \;\leq\; \sum_{i=1}^{j} (d_{n-i+1} - 1), \qquad j = 1, \ldots, n-1.$$

Applying the index reversal $\beta_i := \alpha_{n-i}$ for $i = 1, \ldots, n-1$ rewrites these inequalities as

$$\sum_{\ell=1}^{j} \beta_\ell \;\leq\; \sum_{i=1}^{j} (d_{n-i+1} - 1) = a_j, \qquad j = 1, \ldots, n-1,$$

i.e. the prefix-sum form to which Lemma 5 applies (with n in that lemma replaced by $n-1$). Lemma 5 then bijects these lattice points with lattice paths from $(0, 0)$ to $(n-1, a_{n-1})$ whose i-th horizontal step has height at most a_i. This is exactly the boundary in Theorem 2, so the Dixon matrix size is bounded above by the corresponding lattice-path count, with equality for generic systems under the characteristic hypothesis of Lemma 4.

Comparison with Existing Bounds Previous work [48, Theorem 3.4] presents a Newton polytope characterization of unmixed polynomial systems, whereas [48, Theorem 4.3] provides volume-based bounds (5) for multi-homogeneous systems. These bounds can exhibit some deviation from actual Dixon matrix sizes for large n. In this work, we consider general mixed systems with varying polynomial degrees and establish precise bounds on the Dixon matrix size by formulating it as a lattice path counting problem. The case of $(n; d)$-systems can be viewed as a common specialization of both approaches, where the Fuss-Catalan bound provides the exact value achieved for generic systems. Our formula can also be viewed as a generalization of results in [72] from $d = 2$ to arbitrary degree d.

Appendix C provides a comparison of all bounds and experimental validation confirming that our bounds match actual Dixon matrix sizes for generic systems.

3.2 Complexity Estimates via Our Refined Bound

For clarity of exposition, we focus on $(n; d)$-systems where all polynomials have total degree d. Consider $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{x}, \boldsymbol{a}]$ consisting of n equations in $n-1+k$ variables, where $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ are the eliminated variables and $\boldsymbol{a}$ the retained parameters. Let

$$M := C_n^{(d)} = \frac{1}{n(d-1)+1}\binom{nd}{n}$$

be the generic Dixon matrix size from Corollary 1(2). Since $\operatorname{rank}\mathcal{M} \le D(\mathcal{F})$, the quantity M also upper-bounds the size of the extracted submatrix $\mathcal{M}'$, so the estimates below are upper bounds on the actual cost.

We give a stage-by-stage outline of the relevant model choices below; here we consider the determinant methods with the lowest theoretical complexity or the fastest experimental performance. The detailed parameter derivations, complexity formulas for all five determinant models, and the full Step 1 vs Step 4 comparison are deferred to Appendix H.

Step 1: Compute the cancellation determinant. This step computes det of the $n \times n$ cancellation matrix (1), whose entries are polynomials in $\ell = 2n+k-2$ variables (the $n-1$ original variables, $n-1$ dual variables, and k parameters), with total parameter-degree bounded by $B_1 := nd$. This is the *many-variable, small-matrix* regime, where expansion by minors and sparse interpolation are most competitive. The abstract parameters governing these models admit the following explicit expressions in (n, d, k, M) (full derivation in Appendix H):

$$\mu_1 \le \binom{n-1+k+d}{n-1+k}, \quad S_1 \le M^2\binom{nd+k}{k}, \quad L_1 = \widetilde{O}\left(n^2\binom{n-1+k+d}{n-1+k}\right),$$

where μ_1 is the dense monomial count of one polynomial of total degree d in $n-1+k$ variables, the factor M^2 in S_1 comes from the Dixon bilinear support and $\binom{nd+k}{k}$ from the parameter monomials of total degree at most $B_1 = nd$, and the constant-determinant cost n^ω inside L_1 is absorbed into the $n^2\mu_1$ matrix-evaluation term since $\mu_1 \ge n-1+k$. Substituting (11) into the two preferred determinant models yields

$$T_1^{\text{minor}} = \widetilde{O}(n2^n\mu_1 S_1) = \widetilde{O}\left(n\,2^n\binom{n-1+k+d}{n-1+k}M^2\binom{nd+k}{k}\right), \tag{11}$$

$$T_1^{\text{sparse}} = \widetilde{O}(L_1 S_1 + S_1 \log q) = \widetilde{O}\left(n^2\binom{n-1+k+d}{n-1+k}M^2\binom{nd+k}{k}\right) \tag{12}$$

(the $S_1 \log q$ term in the sparse model is polylogarithmic in the problem size and absorbed into $\widetilde{O}(\cdot)$). The sparse model has the lower asymptotic complexity in the polynomial-in-(n, d) factor (saving $2^n/n$ relative to expansion), but in all our experiments, cached expansion by minors is consistently faster, likely due to the larger constant factors inherent in the sparse interpolation approach.

Step 2: Construct the Dixon matrix. Extracting the Dixon matrix coefficients from the cancellation determinant is dominated by Step 1. An alternative is the recursive construction of Zhao and Fu [76], which builds the Dixon matrix directly via a block recursion; following Qin et al. [62], the resulting surrogate T_2^{FDixon} is attractive when few variables are eliminated and degrees are high, but for many eliminated variables the inherent $(n!)^3$ factor (Appendix H) makes it less competitive than the direct Step 1 path.

Step 3: Extract a maximal-rank submatrix. Step 3 extracts a maximal-rank submatrix numerically after finite-field evaluation, with cost

$$\mathcal{T}_3 = \mathcal{O}(M^\omega). \tag{13}$$

The degree-aware weights of Section 4.1 are computed by a single pass over the M^2 entries, dominated by (13).

Step 4: Compute the Dixon resultant. This step computes the symbolic determinant of $\mathcal{M}'$, a polynomial matrix of size at most M in the k retained parameters $\boldsymbol{a}$. In the Kronecker+HNF model, the complexity is governed by the entry-wise parameter degree $E_4 \le nd$, while in the evaluation-interpolation model it is governed by the total degree D_4 of the elimination polynomial. Assuming that the degree-aware selection of Section 4.1 removes the extraneous factors (Heuristic 2), the elimination polynomial coincides (up to units) with a generator of $\mathcal{I} \cap \mathbb{K}[\boldsymbol{a}]$. By Theorem 1, $D_4 \le d_{\mathcal{I}} \le d^n$. The corresponding parameters are

$$N_4 \le (d^n+1)^k, \qquad L_4 = \widetilde{O}(M^\omega), \qquad \delta_4 = nd\,(M \cdot nd + 1)^{k-1},$$

where δ_4 denotes the average column degree after Kronecker substitution and reduces to $\delta_4 = nd$ when $k = 1$. Therefore,

$$T_4^{\mathrm{HNF}} = \widetilde{O}(M^\omega \delta_4) = \widetilde{O}\big(M^\omega nd(M \cdot nd + 1)^{k-1}\big)\,, \tag{14}$$

$$T_4^{\mathrm{eval}} = \widetilde{O}(N_4(L_4 + kD_4)) = \widetilde{O}\big((d^n+1)^k(M^\omega + k\,d^n)\big)\,. \tag{15}$$

Overall comparison. Let T_1^{best} and T_4^{best} be the minimum cost across all considered models for Steps 1 and 4, respectively. Then $T_{\mathrm{Dixon}} = \widetilde{O}(\max\{T_1^{\mathrm{best}}, T_4^{\mathrm{best}}\})$. At the idealized boundary $\omega = 2$, the two stages have the same asymptotic rate in M. For the practically relevant case $\omega > 2$, T_4^{best} asymptotically dominates as n or d grows, and it also dominates the running time in our experiments. We therefore take T_4^{best} as the overall complexity estimate. A more detailed comparison is given in Appendix H.3.

Well-determined case. For $k = 1$, the final determinant is an $M \times M$ univariate polynomial matrix for which the HNF algorithm of [52] is typically optimal:

$$T_4^{\mathrm{HNF}} = \widetilde{O}(M^\omega\, nd) = \widetilde{O}\left(nd\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^\omega\right). \tag{16}$$

For Step 1, the most competitive models are sparse interpolation and cached expansion by minors; substituting $k = 1$ into (11) and (12) gives

$$T_1^{\mathrm{minor}} = \widetilde{O}\left(n^2\, d\, 2^n \binom{n+d}{n} M^2\right), \tag{17}$$

$$T_1^{\mathrm{sparse}} = \widetilde{O}\left(n^3\, d \binom{n+d}{n} M^2\right). \tag{18}$$

Similar to the general case, due to the rapid growth of $\binom{nd}{n}$, for every $\omega > 2$, the final-determinant cost T_4^{HNF} asymptotically dominates both Step 1 models, and it is also the empirical bottleneck in our experiments. We therefore adopt $T = T_4^{\mathrm{HNF}}$ as the overall complexity estimate for well-determined systems. The full derivation is given in Appendix H.4.

Remark 6. For general $(d_1, \ldots, d_n)$-systems, the same argument yields $T_{\mathrm{Dixon}} = \widetilde{O}\big(\big(\sum_{i=1}^n d_i\big)\, D(\mathcal{F})^\omega\big)$, with $D(\mathcal{F})$ given by Corollary 1(3).

3.3 Comparison with Gröbner Basis

Gröbner basis methods solve polynomial systems by computing a grevlex order basis via F4/F5 [32,31] and then converting to lexicographic order via FGLM [33]. We compare Dixon with these methods on $(n; d)$-systems.

Well-Determined Systems ($m = n$) Numerous studies [32,9,13,67,51] have established theoretical complexity bounds for Gröbner basis algorithms. Table 2 summarizes the complexity formulas for Dixon and various Gröbner basis approaches for $(n; d)$-systems, together with the leading-order asymptotic equivalent of each ratio relative to Dixon.

The two columns are not of the same nature: the Dixon entry is instantiated with the generically attained matrix size M, whereas the Gröbner basis costs are worst-case upper bounds that do not capture practical F4/F5 optimizations, so the ratios compare bounds rather than measured running times. In particular, the FGLM term $\mathcal{O}(n\, d_{\mathcal{I}}^\omega)$ is asymptotically *smaller* than the Dixon cost, since $M = C_n^{(d)}$ exceeds the Bézout number d^n; the Dixon estimate stays below the combined F4/F5+FGLM cost only because F4/F5 is taken as the Gröbner bottleneck. We therefore claim only that the two approaches are comparable on well-determined systems, and rely on the experiments of Section 4.2 for practical comparisons. Figure 1 shows the empirical growth as n varies, and further plots are in Appendix E.

Table 2: Complexity formulas for well-determined $(n; d)$-systems with $d_{\text{reg}} = n(d-1)+1$ and $d_{\mathcal{I}} = d^n$, and ratios relative to Dixon. The "Ratio to Dixon" column reports the leading-order estimate of $\mathcal{T}_{\text{GB}}/\mathcal{T}_{\text{Dixon}}$ for d fixed and $n \to \infty$; these simplified ratios are not uniform in (n, d) (Appendix D), and they compare a generically attained Dixon estimate with worst-case Gröbner basis bounds.

Method	**Complexity**	**Ratio to Dixon**
Dixon Resultant	$\widetilde{\mathcal{O}}\left(nd \cdot \left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}\right)$	1
GB (classical) [32]	$\mathcal{O}\left(\binom{n+d_{\text{reg}}}{d_{\text{reg}}}^{\omega}\right)$	$\mathcal{O}\left((nd)^{\omega-1}\right)$
GB (F5) [9,13]	$\mathcal{O}\left(n \cdot d_{\text{reg}} \cdot \binom{n+d_{\text{reg}}-1}{d_{\text{reg}}}^{\omega}\right)$	$\mathcal{O}\left(n^{\omega+1}\right)$
GB (F5 refined) [9]	$\mathcal{O}\left(n \cdot \binom{n+d_{\text{reg}}-d}{n} \cdot \binom{n+d_{\text{reg}}}{n}^{\omega-1}\right)$	$\mathcal{O}\left(n^{\omega} d^{\omega-1}\right)$
GB (per-deg) [67,51]	$\mathcal{O}\left(n \cdot \sum_{i=0}^{d_{\text{reg}}} \binom{n+i-d-1}{i-d}\binom{n+i-1}{i}^{\omega-1}\right)$	—
FGLM [33,51]	$\mathcal{O}\left(n \cdot d_{\mathcal{I}}^{\omega}\right)$	$\text{poly}(n,d)\left(\frac{d-1}{d}\right)^{(d-1)n\omega}$

The GB (per-deg) row does not admit a closed-form asymptotic equivalent that is both simple and accurate: a leading-order estimate of the index-dominated sum gives only a very loose envelope and disagrees substantially with the experimental F4/F5 cost on this model, so we leave the ratio entry blank and direct attention to the empirical comparison in Figure 1 for this case. Detailed derivations of all ratios are in Appendix D.

Overdetermined Systems ($m > n$) Gröbner basis methods are typically preferable for overdetermined systems. According to [32], the degree of regularity d_{reg} is significantly lower than for well-determined systems, dramatically reducing Macaulay matrix dimensions; overdetermined systems also often yield univariate polynomials directly in the grevlex Gröbner basis, eliminating the FGLM conversion step. In contrast, the Dixon resultant requires exactly n equations to eliminate $n-1$ variables and cannot directly leverage redundancy: when given $m > n$ equations, one selects n equations and verifies the rest afterward, with slightly higher complexity than the well-determined case.

Underdetermined Systems ($m < n$) In the underdetermined setting, from the viewpoint of complete solving, a common cryptanalytic strategy is to guess values for $n-m$ variables and reduce to a (well- or over-)determined subsystem,

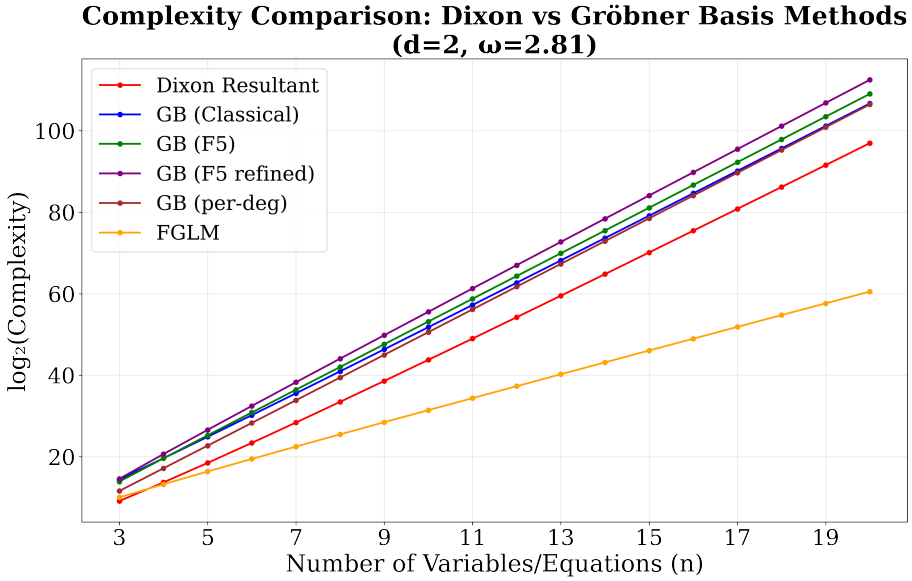

Fig. 1: Complexity comparison varying n with $d = 2$, $\omega = 2$. The y-axis shows $\log_2$(complexity). All Gröbner-basis variants tabulated in Table 2, including the GB (per-degree) bound, are included for direct empirical comparison with Dixon.

which is then solved by Gröbner bases. We do not propose Dixon as a replacement here: solving a positive-dimensional ideal in full generality is usually best handled by guess-and-reduce when one solution suffices.

Instead, from the viewpoint of elimination, Dixon provides a flexible determinant-based approach: from a system with more variables, it removes several variables at once and produces explicit relations in fewer variables, which can then be combined with guessing or meet-in-the-middle decompositions. By contrast, changing term orders for positive-dimensional Gröbner basis computations typically requires Gröbner walks [25], whose complexity is hard to predict in general.

In summary, the two methods are complementary: Dixon and Gröbner basis are comparable on well-determined systems; Gröbner basis is typically preferable for overdetermined systems; Dixon is attractive for underdetermined systems as a determinant-based elimination method.

4 Optimized Implementation for Dixon Resultant

While the theoretical complexity analysis highlights the advantages of Dixon resultant methods, bridging the gap to practical efficiency requires overcoming several significant hurdles. Currently, off-the-shelf computer algebra systems

offer limited support: Dixon resultants are not natively implemented in some advanced solvers like Magma [20], and while implementations exist in Fermat [53] and through third-party packages in Maple [46], their performance is often unsatisfactory at scale. Beyond software limitations, extraneous factors must be carefully controlled or the classical Bézout bound becomes inapplicable, and large-scale polynomial-matrix operations must be handled efficiently. This section addresses these issues.

4.1 Eliminating Extraneous Factors via Degree-Aware Submatrix Selection

The primary challenge in implementing the Dixon resultant arises from the *extraneous factors* of Definition 2, which do not correspond to actual solutions of the original system and can substantially inflate the resultant degree beyond the Bézout bound (Theorem 1). Without extraneous factors, the degree is controlled by the ideal degree and hence by Bézout, providing both theoretical elegance and computational efficiency.

Qin et al. [63] proposed a method based on parameter specialization and regular-chain computation; however, the regular-chain step incurs additional overhead. In contrast, we propose a *preventive* strategy that minimizes extraneous factors during the matrix construction phase rather than eliminating them afterwards. Our experimental results show that the proposed greedy selection algorithm reduces redundant factors, and in most cases the resulting Dixon determinants remain at or below the Bézout bound.

Degree-aware submatrix selection strategy. A naive constant-evaluation rank test ignores the degree structure of the resulting submatrix. We develop a degree-aware selection strategy that employs a greedy approach to prioritize rows and columns with lower degrees. Following the determinant degree estimation framework of Labahn et al. [52], for any non-singular polynomial matrix A,

$$\deg(\det(A)) \leq \min(|\mathrm{rdeg}(A)|, |\mathrm{cdeg}(A)|),$$

where $|\mathrm{rdeg}(A)|$ and $|\mathrm{cdeg}(A)|$ denote the sum of row degrees and column degrees, respectively. Our method is based on the following working hypothesis:

Heuristic 1 (Degree Minimization Heuristic) *For two polynomial matrices A and B of the same dimension over $\mathbb{K}[t_1, \ldots, t_m]$, if $\min(|rdeg(A)|, |cdeg(A)|) > \min(|rdeg(B)|, |cdeg(B)|)$, then we expect that $\deg(\det(A)) > \deg(\det(B))$ holds in the generic case.*

The intuition is that generic determinants saturate their row/column-sum upper bound, so a smaller bound usually corresponds to a smaller actual degree. Under this hypothesis, our greedy strategy yields maximal-rank submatrices with minimal determinant degree, thereby reducing extraneous factors in the resulting Dixon resultant.

Remark 7. Labahn et al. [52] also utilized a refined bound $\deg(\det(A)) \leq \max_{\pi \in S_n} \sum_i \deg(a_{i,\pi(i)})$, where S_n is the symmetric group on $\{1, \ldots, n\}$. This enables deterministic selection of submatrices with minimal determinant degree, but its factorial cost makes it infeasible for large n. Our experiments indicate that Heuristic 1 is sufficient to achieve minimal determinant degrees in practice, although it does not hold rigorously.

Degree-aware selection algorithm. Based on Heuristic 1, we design a greedy algorithm that combines rank verification with degree-aware ordering of rows and columns. The core idea is to assign degree weights to rows and columns of the Dixon matrix and prioritize those with lower degrees during Gaussian elimination, ensuring the selected submatrix has maximal rank while heuristically favoring a low determinant degree. The full pseudocode is in Appendix F.

The rank test is performed at a random evaluation point, so the identified rank equals the true symbolic rank only with high probability; by Schwartz–Zippel lemma [66,77], the failure probability is negligible, and a complete analysis in the same setting can be found in [47,45].

Heuristic 2 *Let $\mathcal{P} = \{p_1, \ldots, p_n\}$ be a generic polynomial system with degrees $d_1, \ldots, d_n$, and $\mathcal{M}$ its Dixon matrix. The submatrix selected by Algorithm 1 yields a resultant R satisfying $\deg(R) \leq \prod_{i=1}^{n} d_i$.*

This is supported by computational experiments (Appendix G). In very rare cases where the heuristic does not fully eliminate every extraneous factor, the observed degree of R exceeds the Bézout bound by a small constant, which does not affect the asymptotic estimates of Section 3.2. Note that comparing $\deg(R)$ with the Bézout bound is only a test, not a proof that $\det(\mathcal{M}')$ generates $\mathcal{I} \cap \mathbb{K}[\boldsymbol{a}]$; the bounds using $D_4 \leq d_{\mathcal{I}}$ are stated under this heuristic.

4.2 Implementation

We developed a comprehensive, open-source C implementation of the Dixon resultant method, `DRSolve`, built upon the high-performance `FLINT` and `PML` libraries (parts of the code were written with the help of AI coding assistants, as disclosed in the supplementary `README`), available at https://github.com/drsolve/drsolve. It supports operations over prime fields $\mathbb{F}_p$ of arbitrary size, general finite fields $\mathbb{F}_q$, and $\mathbb{Q}$. It incorporates most Dixon construction techniques from [54,76,45] and most polynomial-matrix determinant techniques from [52,44,43,56]. It provides:

- **Dixon resultant elimination:** Eliminate $n - 1$ variables from n polynomials using the Dixon resultant.
- **Polynomial system solving:** Compute all solutions of zero-dimensional systems.
- **Flexible determinant algorithms:** Support for all determinant computation methods discussed in this paper.

– **Optimized binary arithmetic:** Highly optimized for binary extension fields $\mathbb{F}_{2^k}$ ($k \in \{8, 16, 32, 64, 128\}$), common in cryptographic applications.
– **Multi-modular arithmetic:** Support for multi-modular techniques with CRT-based reconstruction over $\mathbb{Q}$ and large prime fields.
– **Complexity estimation:** A tool for estimating the cost of a polynomial system, or based on parameters such as the number of variables and degrees.

Experimental results. All benchmarks were performed on a dual-socket AMD EPYC 7302 server. We compared our Dixon implementation with the Gröbner basis solvers in Magma [20] and msolve [14]. As shown in Figure 2, the results reveal a clear trade-off: Magma performs best for systems with more variables and lower degrees, possibly benefiting from F4/F5 optimizations, while `DRSolve` shows significant advantages for systems with fewer variables and higher degrees. We also compared `DRSolve` with the closest Dixon software: on dense 3-variable systems of degree 8 over $\mathbb{F}_p$ it terminates in about 0.1 s, whereas Dixon-EDF inside Fermat [53,54], the fastest prior implementation we could run, needs about 15 s; the Maple+C code of Jinadu and Monagan [46,45] targets parametric systems over $\mathbb{Q}$ and is not directly comparable in our setting.

5 Application to AO Primitives

This section organizes the applications around three elimination patterns. The goal is to highlight the solving methodology first and to use concrete AO primitives as representative case studies. We focus on solving strategies rather than on new algebraic models for AO primitives, reusing the best-known models from the literature and reevaluating them through the Dixon-resultant viewpoint.

Remark 8. Although much of the preceding discussion is developed under genericity assumptions, the associated structural bounds—such as degree bounds, Dixon matrix size estimates, and determinant complexity estimates—remain valid as general upper bounds even for non-generic polynomial systems arising in AO primitives. While these bounds may not be tight in structured instances, they still provide a consistent complexity envelope for algebraic analysis.

5.1 Method 1: Direct elimination

Method 1 applies the Dixon resultant directly to a square polynomial system. Starting from n equations in n variables, we eliminate $n-1$ variables, solve the resulting elimination polynomial in the remaining parameter, and then back-substitute recursively. The whole algebraic burden is concentrated in one structured determinant, so the complexity analysis reduces to the matrix-size and determinant models from Section 3.2. This is the classical Dixon solving workflow of [72,3], instantiated here with the bounds of Section 3 and the implementation of Section 4. This method is most suitable when the primitive admits a dense but still manageable algebraic description, so that the later back-substitution phases operate on lower-dimensional systems and the first elimination dominates.

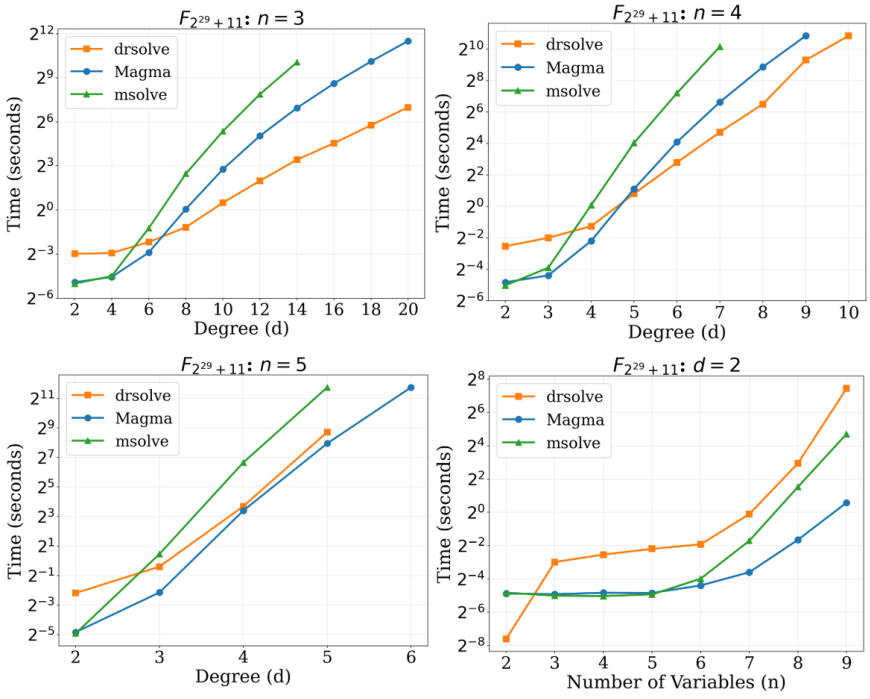

Fig. 2: Performance comparison (time in seconds) of Gröbner basis methods (Magma 2.29-7 and msolve 0.9.5) versus our `DRSolve` 0.3.0 on randomly generated dense polynomial systems over $\mathbb{F}_p$. Here n denotes the number of variables (equal to the number of equations).

This method is best suited to Type I (low-degree) primitives, whose CICO systems admit a dense but uniformly low-degree algebraic description; we take Poseidon as the representative example.

Application to Poseidon. We consider the CICO problems of Poseidon [38] and Poseidon2 [39] in sponge mode, using the input/output and internal-subspace models from [40]; the precise models are summarized in Appendix J. For the pure I/O model, direct Dixon elimination matches the natural square-system viewpoint. For the mixed-degree subspace models, the determinant bound from Section 3 becomes essential because the polynomial degrees are no longer uniform. Figure 3 compares the resulting complexities for $t = 8$, $c = d = 3$, $\alpha = 3$ across different numbers of partial rounds.

Table 3 reports small-instance timings. Full-round systems behave close to generic dense systems, while partial-round systems are much more structured, so both the Dixon matrix size and the Gröbner-basis degree of regularity fall well below their generic upper bounds. Tight a-priori bounds for non-generic partial-round systems are hard since the drop depends on the specific subspace

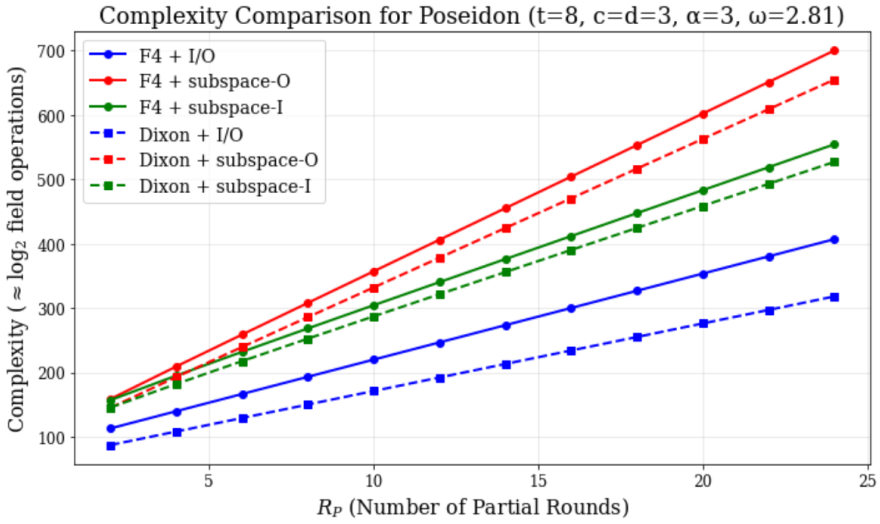

Fig. 3: Complexity comparison for Poseidon and Poseidon2 ($t = 8$, $c = d = 3$, $\alpha = 3$). Here $\omega = 2.81$ for all curves.

model; our generic bound (and the corresponding Figure 3 curves on partial-round regimes) should therefore be read as an upper-bound template rather than a tight prediction of the best attack.

5.2 Method 2: Iterative elimination

Throughout this section calligraphic letters denote blocks: $\mathcal{F}, \mathcal{P}, \mathcal{R}$ are blocks of polynomials, $\mathcal{X}, \mathcal{Z}$ are blocks of variables, and $|\mathcal{X}|$ is the number of variables in the block $\mathcal{X}$.

Method 2 is designed for ladder-structured systems and generalizes iterative two-polynomial elimination to local multipolynomial subsystems. It can be viewed as a generalization of the resultant-based approach of Yang et al. [75] to multipolynomial Dixon elimination, and as an AO-oriented specialization of hybrid Dixon-matrix construction from [28]. We write such a system as

$$\mathcal{S}: \quad \mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1),\ \mathcal{F}_2(\mathcal{X}_1, \mathcal{X}_2),\ \ldots,\ \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{X}_n), \tag{19}$$

with $|\mathcal{F}_1| = |\mathcal{X}_0| + 1$ and $|\mathcal{F}_i| = |\mathcal{X}_{i-1}|$ for $2 \le i \le n$. Eliminating successively,

$$R_1 = \mathrm{Res}_{\mathcal{X}_0}(\mathcal{F}_1),\ R_2 = \mathrm{Res}_{\mathcal{X}_1}(R_1, \mathcal{F}_2),\ \ldots,\ R_n = \mathrm{Res}_{\mathcal{X}_{n-1}}(R_{n-1}, \mathcal{F}_n), \tag{20}$$

yields a univariate eliminant if $|\mathcal{X}_n| = 1$, or a smaller system otherwise.

Assume each block $\mathcal{F}_j$ has common degree d_j. By Bézout, the ideal-theoretic eliminant produced at step i satisfies

$$\deg(R_i) \le \prod_{j=1}^{i} d_j^{|\mathcal{F}_j|}. \tag{21}$$

Table 3: Timing comparison for Poseidon instances ($t = 6, c = 1$, I/O modeling, bypass one round) over $\mathbb{F}_p$, $p = 2^{62} + 169$. T and E denote Theoretical and Experimental values; complexities (Comp.) are in $\log_2$ field operations. Here $\omega = 2.81$. For odd R_F, one additional full round is added at the beginning. Both the Comp. and the timed implementation use the HNF determinant model.

Configuration	**Gröbner Basis**					**Dixon Resultant**				
	d_{reg}		Comp.		Time	$D(F)$		Comp.		Time
	T	E	T	E	(s)	T	E	T	E	(s)
$R_F = 3, R_P = 0, \alpha = 3$	25	25	32.8	32.8	2.2	117	108	24.1	23.7	0.3
$R_F = 3, R_P = 0, \alpha = 5$	73	71	45.2	44.9	94295	925	875	33.9	33.7	1477
$R_F = 4, R_P = 0, \alpha = 3$	79	79	46.2	46.2	188189	1080	1053	34.7	34.6	1772
$R_F = 2, R_P = 1, \alpha = 3$	25	11	32.8	23.9	0.02	117	36	24.1	19.3	0.006
$R_F = 2, R_P = 1, \alpha = 5$	73	27	45.2	33.7	14.3	925	175	33.9	27.2	2.1
$R_F = 3, R_P = 1, \alpha = 3$	79	31	46.2	35.3	178	1080	891	34.7	33.9	695
$R_F = 2, R_P = 2, \alpha = 3$	79	24	46.2	32.4	0.4	1080	297	34.7	29.4	11.9

The last step produces R_n from R_{n-1} and $\mathcal{F}_n$, by eliminating the variables in $\mathcal{X}_{n-1}$. Here R_{n-1} has degree $D := \prod_{j=1}^{n-1} d_j^{|\mathcal{F}_j|}$ and each polynomial in $\mathcal{F}_n$ has degree d_n. Viewing this as Dixon elimination on one polynomial of degree D and $|\mathcal{F}_n|$ polynomials of degree d_n, the Unrestricted Bound (6) gives matrix size at most $\binom{d_n|\mathcal{F}_n| - d_n + D}{|\mathcal{F}_n|}$. The total $\mathcal{X}_n$-degree of the cancellation determinant is at most $D + d_n|\mathcal{F}_n|$ (sum of the per-row degrees in the eliminated block), which in turn upper-bounds the entry-wise $\mathcal{X}_n$-degree of the resulting Step 4 polynomial matrix (Section 3.2 Step 4), so HNF gives

$$\mathcal{O}\left((D + d_n|\mathcal{F}_n|) \binom{d_n|\mathcal{F}_n| - d_n + D}{|\mathcal{F}_n|}^{\omega} \right), \tag{22}$$

with the Determinant Bound (8) available when the Unrestricted Bound is too coarse.

For meet-in-the-middle (MITM) instances, we split the chain into forward and backward parts meeting in a small middle block $\mathcal{Z}$:

$$\mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1), \ldots, \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{Z}), \quad \mathcal{F}'_1(\mathcal{Y}_0, \mathcal{Y}_1), \ldots, \mathcal{F}'_m(\mathcal{Y}_{m-1}, \mathcal{Z}). \tag{23}$$

Forward and backward eliminations give two middle-state relations of degrees

$$D_{\text{fwd}} = \prod_{j=1}^{n} d_j^{|\mathcal{F}_j|}, \qquad D_{\text{bwd}} = \prod_{j=1}^{m} d'_j{}^{|\mathcal{F}'_j|}.$$

The final merge eliminates the middle block $\mathcal{Z}$ and, with $d_{\max} = \max(D_{\text{fwd}}, D_{\text{bwd}})$, admits the coarse bound

$$\mathcal{O}\left(d_{\max}^{\omega+1}\right), \tag{24}$$

obtained as follows. We arrange the split so that the middle block $\mathcal{Z}$ consists of two shared variables, with the forward chain contributing one relation in $\mathcal{Z}$ and the backward chain contributing another, of total degrees D_{fwd} and D_{bwd} respectively. The merge is then a resultant within $\mathcal{Z}$: it eliminates one of the two middle variables between these two relations, leaving the final univariate polynomial in the remaining middle variable. The associated Dixon matrix has size $\Theta(d_{\max})$ (driven by the degree of the two middle relations in the eliminated variable), and each of its entries is a polynomial of degree at most $2d_{\max}$ in the retained variable (a Dixon entry is a difference-quotient combination of two row entries, each of degree $\leq d_{\max}$). The univariate HNF determinant on such a matrix costs $\widetilde{O}(d_{\max}^{\omega} \cdot 2d_{\max}) = \widetilde{O}(d_{\max}^{\omega+1})$ (Step 4 with matrix size $\Theta(d_{\max})$ and average column degree $\Theta(d_{\max})$, $k = 1$).

The complexity of Method 2 is the maximum over all stages: every determinant of the forward and backward chains (22), the final merge (24), univariate root finding, back-substitution, and the verification of the candidate solutions in the original system, which also discards the spurious candidates possibly introduced by extraneous factors. The verification cost is negligible, and for the parameters reported below the final merge is the dominant stage.

This method is best suited to Type II (equivalence-relation) primitives, whose ladder-structured CICO systems link successive states through local low-degree relations; we take `Vision` as the representative example.

Application to Vision. For `Vision` [4] with $s = 2$, inverse layers make the full direct model unattractive, so we adopt a meet-in-the-middle (MITM) approach following the general framework of Liu et al. [57] and apply iterative elimination with Dixon resultant. The detailed model and equation distribution are in Appendix K; Figure 4 and Table 4 summarize the outcome.

The theoretical MITM bound still grows exponentially in the number of rounds, but measured behavior is milder because many quadratic equations do not contribute substantially to degree growth. This is the structured regime where local elimination is preferable to direct elimination.

Table 4: Experimental timing and complexity comparison for `Vision` ($s = 2$). Complexities (Comp.) are in $\log_2$ field operations. $\omega = 2$ for consistency with [57].

	2 rounds		**3 rounds**		**4 rounds**	
Method	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis [57]	31.0	1.38s	42.0	1d 4h	53.0	—
Dixon resultant	25.0	0.1s	36.0	15m	49.0	3d

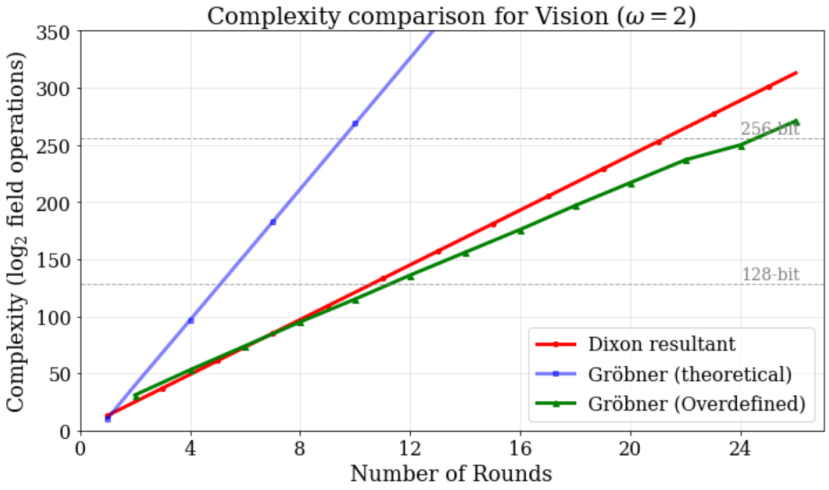

Fig. 4: Complexity comparison for `Vision` ($\omega = 2$ for consistency with [57].).

5.3 Method 3: Ideal reduction and hybrid elimination

Method 3 combines Dixon elimination with the reduction machinery developed for FreeLunch-style systems in [11]. Starting from a triangular system

$$\mathcal{P} = \begin{cases} z_1^\alpha - f_1(x) = 0, \\ z_2^\alpha - f_2(x, z_1) = 0, \\ \vdots \\ z_n^\alpha - f_n(x, z_1, \ldots, z_{n-1}) = 0, \\ h(x, z_1, \ldots, z_n) = 0, \end{cases} \tag{25}$$

and use the reduction procedure `Reduce` of [11, Algorithm 1]. For a polynomial g of weighted degree $d_{\mathcal{P}_k}(g)$ in a single retained variable, the reduction cost is $\widetilde{O}(d_{\mathcal{P}_k}(g)(2\alpha - 1)^n)$, where n is the number of levels in the triangular ideal $\mathcal{P}_k$. For CICO-1, the optimized two-polynomial strategy of [11] applies directly and gives

$$\widetilde{O}\left(d_I \alpha^2 \left(\frac{2\alpha - 1}{\alpha}\right)^{n+2}\right). \tag{26}$$

For general CICO-k we retain $|\mathcal{X}| = k$ input variables and build k relations $R_1(\mathcal{X}), \ldots, R_k(\mathcal{X})$ in three phases.

(P1) Reduction. Each output constraint h_j is replaced by its normal form modulo the triangular ideal, using [11, Algorithm 1]. Since the polynomials manipulated here are k-variate in $\mathcal{X}$, the coefficient-count factor of [11, Lemma 1] is replaced by the dense coefficient count $B_k(D) := \binom{D+k}{k} = \Theta(D^k)$ of a k-variate polynomial of degree D. Reduction never increases the weighted degree, so with

$D_0 := \max_j d_{\mathcal{P}_k}(h_j)$ this phase costs

$$T_{\text{red}} = \widetilde{O}(k\, B_k(D_0)\, (2\alpha - 1)^n)\,. \tag{27}$$

(P2) Successive elimination. Reduction only lowers the exponents $\deg_{z_i}$ below α; it does not by itself produce an element of $\mathbb{K}[\mathcal{X}]$. The remaining powers of the z_i are removed by successive resultants, each followed by a reduction as in [11, Proposition 7], with the same substitution of $B_k(\cdot)$ for the coefficient count. Following the degree-growth bound of [11, Proposition 7], let $D_i \leq \alpha^{n-i} D_0$ denote the weighted-degree bound at the stage with i triangular levels remaining. Summing over all stages and all k constraints,

$$T_{\text{elim}} = \widetilde{O}\left(k \sum_{i=1}^{n} B_k(D_i)\, (2\alpha - 1)^{i+2}\right). \tag{28}$$

(P3) Final elimination. Let $D := \max_j \deg(R_j)$. The final step is a mixed-degree Dixon elimination on $\{R_1, \ldots, R_k\}$, i.e. k polynomials in k variables of which $k-1$ are eliminated:

$$T_{\text{final}} = \widetilde{O}(kD\, (D(\mathcal{R}))^{\omega})\,, \tag{29}$$

with $D(\mathcal{R})$ given by Corollary 1(3), which specializes to $C_k^{(D)}$ when all the R_j have the same degree.

The total hybrid cost is the maximum of (27)–(29); in particular the reduction and elimination phases are not absorbed into the final Dixon cost in general. By contrast, direct elimination on the unreduced system of $n+k$ equations gives

$$\widetilde{O}\left(\left(\sum_i d_i\right) (D(\mathcal{P}_k))^{\omega}\right), \tag{30}$$

again with $D(\mathcal{P}_k)$ from Corollary 1(3), since the constraints h_j do not have the same degree as the triangular equations; it reduces to $\mathcal{O}\Big((n+k)\alpha(C_{n+k}^{(\alpha)})^{\omega}\Big)$ when all equations have degree α. The comparison between the two routes determines whether the reduction phase is worthwhile.

This method is an improved attack on Type II (equivalence-relation) primitives, refining Method 2 by interleaving Dixon elimination with ideal reduction; we take `XHash12` as the representative example.

Remark 9. For the special case of CICO-1 the two-polynomial iterative-resultant route of [11] already gives (26) and Dixon does not change the picture; we use Dixon here precisely for CICO-k with $k \geq 2$, where the final elimination step combines k reduced relations and is no longer of the two-polynomial form addressed by [11]. Although Method 3 does not yet provide concrete advantages for instances beyond CICO-3, the framework is substantially more flexible and opens additional directions for future elimination strategies.

Application to XHash12. We focus on `XHash12` [5]. Figure 5 compares the direct Dixon method with the hybrid reduction-based strategy for $k = 1, 2, 3, 4$. The hybrid method is excellent for small-k instances, because the reduction phase keeps the final elimination degrees low. For the natural CICO-4 setting, however, its intermediate degrees become too large; in that regime the direct Dixon elimination remains the more robust option.

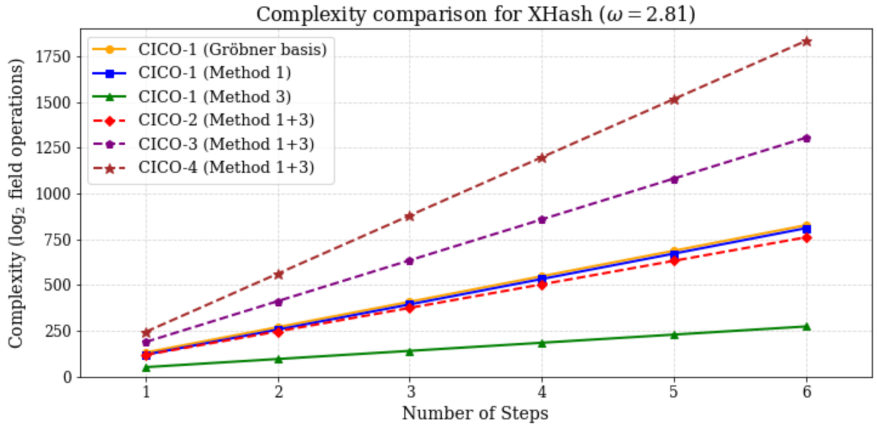

Fig. 5: Complexity comparison for `XHash12`, $\omega = 2.81$.

This picture is confirmed by the small-instance timings in Table 5. In short: Method 3 explains why CICO-1 instances can be unusually easy, but Method 1 gives the more relevant estimate for the standard XHash12 security parameters. The example also illustrates a broader point: hybrid elimination is powerful only when the reduction phase lowers the degrees faster than it multiplies the number or complexity of the remaining relations. Note that for CICO-2 (2 step), Method 1+3 has lower asymptotic complexity (31.2) than Method 1 (40.7) but a slower measured time, because for small instances the reduction-phase constants dominate the leading-order term.

Table 5: Experimental time comparison for small `XHash12` instances with $c = 1$, where $\alpha = 3$ for the two-step instances and $\alpha = 7$ for the three-step instance. The complexities (Comp.) are in $\log_2$ field operations. $\omega = 2.81$.

Method	CICO-1 (2 step)		CICO-1 (3 step)		CICO-2 (2 step)	
	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis	41.4	1.5s	63.3	-	48.8	59.7s
Method 1	35.1	9.4s	64.4	-	40.7	4.5s
Method 1+3	18.6	0.04s	29.6	200.4s	31.2	413.2s

6 Conclusion

In this work, we revisited the Dixon resultant as a practical and theoretically grounded tool for algebraic cryptanalysis. By establishing upper bounds on the Dixon matrix size that are attained for generic systems, we derived refined complexity estimates. Combined with an efficient open-source implementation and a degree-aware submatrix selection strategy, our framework enables reliable elimination for polynomial systems arising in cryptographic applications.

Our results show that Dixon resultants and Gröbner basis methods play complementary roles in algebraic cryptanalysis. In well-determined settings, Dixon resultants are comparable to Gröbner bases. Gröbner basis methods are more effective for overdetermined systems, whereas Dixon resultants provide a structured determinant-based elimination subroutine for under-determined settings.

There remain several directions for future work.

1. Developing algebraic models and hybrid elimination strategies for AO primitives, including higher-order CICO-k systems, and for Type III (lookup-table) constructions with recent algebraic techniques [8];
2. Improving Dixon-resultant techniques, including nonheuristic algorithms for extraneous-factor elimination, exploration of more advanced interpolation strategies, and possible connections with multivariate subresultants [27];
3. Extending the framework to multivariate public-key schemes such as UOV and Rainbow. Direct Dixon elimination is asymptotically more efficient than naive Gröbner basis methods for MQ systems but still less effective than hybrid strategies [18], yet we can investigate whether Method 2/3 can be combined with specialized strategies such as [59] for MQ systems.

Acknowledgements. We would like to thank our advisors, Meiqin Wang and Kai Hu, for their guidance and support. We are grateful to all the anonymous reviewers for their valuable comments and suggestions. We thank Scarborough for assistance with the proof of Lemma 2 in Appendix B. We also thank Robert H. Lewis, Michael Monagan, Ayoola Jinadu, and Guoqiang Deng for their help with the implementation of the Dixon resultant.

Claude and ChatGPT were used to assist with the implementation of the DRSolve software. All proofs, algorithms, and mathematical derivations were developed by the authors. All AI-assisted code was reviewed and tested by the authors.

References

1. Albrecht, M.R., Grassi, L., Perrin, L., Ramacher, S., Rechberger, C., Rotaru, D., Roy, A., Schofnegger, M.: Feistel structures for mpc, and more. In: ESORICS 2019. Lecture Notes in Computer Science, vol. 11736, pp. 151–171. Springer (2019). https://doi.org/10.1007/978-3-030-29962-0_8
2. Albrecht, M.R., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: ASIACRYPT 2016. Lecture Notes in Computer Science, vol. 10031, pp. 191–219 (2016). https://doi.org/10.1007/978-3-662-53887-6_7

3. Albrecht, M.R.: Algebraic Attacks on the Courtois Toy Cipher. Master's thesis, Universität Bremen (2006), tO BE VERIFIED: title/type/institution and the chapter on the Dixon resultant
4. Aly, A., Ashur, T., Ben-Sasson, E., Dhooghe, S., Szepieniec, A.: Design of symmetric-key primitives for advanced cryptographic protocols. IACR Trans. Symmetric Cryptol. **2020**(3), 1–45 (2020). https://doi.org/10.13154/tosc.v2020.i3.1-45
5. Ashur, T., Kindi, A., Mahzoun, M.: Xhash8 and xhash12: Efficient stark-friendly hash functions. IACR Cryptol. ePrint Arch. p. 1045 (2023), https://eprint.iacr.org/2023/1045
6. Aval, J.: Multivariate fuss-catalan numbers. Discret. Math. **308**(20), 4660–4669 (2008). https://doi.org/10.1016/J.DISC.2007.08.100
7. Bak, A., Bariant, A., Boeuf, A., Briaud, P., Øygarden, M., Phanse, A.: The algebraic cheaplunch: Extending freelunch attacks on arithmetization-oriented primitives beyond CICO-1. IACR Cryptol. ePrint Arch. p. 2040 (2025), https://eprint.iacr.org/2025/2040
8. Bak, A., Jazeron, G., Galissant, P., Perrin, L.: Attacking split-and-lookup-based primitives using probabilistic polynomial system solving applications to round-reduced monolith and full-round skyscraper. IACR Trans. Symmetric Cryptol. **2025**(3), 337–367 (2025). https://doi.org/10.46586/TOSC.V2025.I3.337-367, https://doi.org/10.46586/tosc.v2025.i3.337-367
9. Bardet, M., Faugère, J., Salvy, B.: On the complexity of the F5 gröbner basis algorithm. J. Symb. Comput. **70**, 49–70 (2015). https://doi.org/10.1016/J.JSC.2014.09.025
10. Bareiss, E.H.: Sylvester's identity and multistep integer-preserving gaussian elimination. Mathematics of Computation **22**(103), 565–578 (1968). https://doi.org/10.1090/S0025-5718-1968-0226829-0
11. Bariant, A., Boeuf, A., Briaud, P., Hostettler, M., Øygarden, M., Raddum, H.: Improved resultant attack against arithmetization-oriented primitives. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V. Lecture Notes in Computer Science, vol. 16004, pp. 335–367. Springer (2025). https://doi.org/10.1007/978-3-032-01901-1_11
12. Bariant, A., Boeuf, A., Lemoine, A., Ayala, I.M., Øygarden, M., Perrin, L., Raddum, H.: The algebraic freelunch: Efficient gröbner basis attacks against arithmetization-oriented primitives. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 14923, pp. 139–173. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_5
13. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. IACR Trans. Symmetric Cryptol. **2022**(3), 73–101 (2022). https://doi.org/10.46586/TOSC.V2022.I3.73-101
14. Berthomieu, J., Eder, C., Safey El Din, M.: msolve: A library for solving polynomial systems. In: Proceedings of the 2021 on International Symposium on Symbolic and Algebraic Computation. pp. 51–58. ISSAC '21, ACM (2021). https://doi.org/10.1145/3452143.3465545
15. Berthomieu, J., Neiger, V., Din, M.S.E.: Faster change of order algorithm for Gröbner bases under shape and stability assumptions. In: Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation, ISSAC

2022. pp. 409–418. ACM (2022). https://doi.org/10.1145/3476446.3535484, https://hal.sorbonne-universite.fr/hal-03580736
16. Bertoni, G., Daemen, J., Peeters, M., Assche, G.V.: On the indifferentiability of the sponge construction. In: Smart, N.P. (ed.) Advances in Cryptology - EUROCRYPT 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. Proceedings. Lecture Notes in Computer Science, vol. 4965, pp. 181–197. Springer (2008). https://doi.org/10.1007/978-3-540-78967-3_11
17. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. In: ECRYPT hash workshop. vol. 2007 (2007)
18. Bettale, L., Faugère, J., Perret, L.: Hybrid approach for solving multivariate systems over finite fields. J. Math. Cryptol. **3**(3), 177–197 (2009). https://doi.org/10.1515/JMC.2009.009, https://doi.org/10.1515/JMC.2009.009
19. Bona, M.: Handbook of Enumerative Combinatorics. Discrete Mathematics and Its Applications, CRC Press (2015), https://books.google.com.tw/books?id=j3kZBwAAQBAJ
20. Bosma, W., Cannon, J., Playoust, C.: The Magma Algebra System I: The User Language (1997). https://doi.org/10.1006/jsco.1996.0125
21. Bouvier, C.: Cryptanalysis and design of symmetric primitives defined over large finite fields. (Cryptanalyse et conception de primitives symétriques définies sur de grands corps finis). Ph.D. thesis, Sorbonne University, Paris, France (2023), https://tel.archives-ouvertes.fr/tel-04327955
22. Bouvier, C., Briaud, P., Chaidos, P., Perrin, L., Salen, R., Velichkov, V., Willems, D.: New design techniques for efficient arithmetization-oriented hash functions: ttanemoi permutations and ttjive compression mode. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14083, pp. 507–539. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_17
23. Cahill, N.D., D'Errico, J.R., Narayan, D.A., Narayan, J.Y.: Fibonacci determinants. The college mathematics Journal **33**(3), 221–225 (2002)
24. Chtcherba, A.D., Kapur, D.: Constructing Sylvester-type resultant matrices using the Dixon formulation. J. Symb. Comput. **38**(1), 777–814 (2004). https://doi.org/10.1016/j.jsc.2004.02.002, tO BE VERIFIED: pages and DOI
25. Collart, S., Kalkbrener, M., Mall, D.: Converting bases with the gröbner walk. J. Symb. Comput. **24**(3/4), 465–469 (1997). https://doi.org/10.1006/JSCO.1996.0145
26. Cox, D., Little, J., O'Shea, D.: Using Algebraic Geometry. Graduate Texts in Mathematics, Springer New York (2005), https://books.google.com.tw/books?id=1blxizOS9N0C
27. Cox, D., D'andrea, C.: Subresultants and the shape lemma. Math. Comput. **92**(343), 2355–2379 (2023). https://doi.org/10.1090/MCOM/3840
28. Deng, G., Qi, N., Tang, M., Duan, X.: Constructing dixon matrix for sparse polynomial equations based on hybrid and heuristics scheme. Symmetry **14**(6), 1174 (2022). https://doi.org/10.3390/sym14061174
29. Dixon, A.L.: The eliminant of three quantics in two independent variables. Proceedings of the London Mathematical Society **s2-7**(1), 49–69 (1909). https://doi.org/https://doi.org/10.1112/plms/s2-7.1.49, https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-7.1.49

30. Faugere, J.C.: A new efficient algorithm for computing gröbner bases (f4). Journal of pure and applied algebra **139**(1-3), 61–88 (1999)
31. Faugère, J.C.: A new efficient algorithm for computing gröbner bases without reduction to zero (f5). In: Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation. p. 7583. ISSAC '02, Association for Computing Machinery, New York, NY, USA (2002). https://doi.org/10.1145/780506.780516
32. Faugère, J.C., Bardet, M., Salvy, B.: On the complexity of gröbner basis computation of semi-regular overdetermined algebraic equations (2004), https://api.semanticscholar.org/CorpusID:7982329
33. Faugère, J., Gianni, P.M., Lazard, D., Mora, T.: Efficient computation of zero-dimensional gröbner bases by change of ordering. J. Symb. Comput. **16**(4), 329–344 (1993). https://doi.org/10.1006/JSCO.1993.1051
34. Gentleman, W.M., Johnson, S.C.: Analysis of algorithms, a case study: Determinants of matrices with polynomial entries. ACM Transactions on Mathematical Software **2**(3), 232–241 (1976)
35. Grassi, L., Hao, Y., Rechberger, C., Schofnegger, M., Walch, R., Wang, Q.: Horst meets fluid-SPN: Griffin for zero-knowledge applications. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023. Lecture Notes in Computer Science, vol. 14083, pp. 573–606. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_19
36. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Reinforced concrete: A fast hash function for verifiable computation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022. pp. 1323–1335. ACM (2022). https://doi.org/10.1145/3548606.3560686
37. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Monolith: Circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. IACR Trans. Symmetric Cryptol. 2024(3); Cryptology ePrint Archive, Paper 2023/1025 (2024), https://eprint.iacr.org/2023/1025
38. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M.D., Greenstadt, R. (eds.) 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021. pp. 519–535. USENIX Association (2021), https://www.usenix.org/conference/usenixsecurity21/presentation/grassi
39. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A faster version of the poseidon hash function. In: Mrabet, N.E., Feo, L.D., Duquesne, S. (eds.) Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14064, pp. 177–203. Springer (2023). https://doi.org/10.1007/978-3-031-37679-5_8
40. Grassi, L., Koschatko, K., Rechberger, C.: Poseidon and neptune: Gröbner basis cryptanalysis exploiting subspace trails. IACR Trans. Symmetric Cryptol. **2025**(2), 34–86 (2025). https://doi.org/10.46586/TOSC.V2025.I2.34-86
41. Grassi, L., Onofri, S., Pedicini, M., Sozzi, L.: Invertible quadratic non-linear layers for mpc-/fhe-/zk-friendly schemes over fnp application to poseidon. IACR Trans. Symmetric Cryptol. **2022**(3), 20–72 (2022). https://doi.org/10.46586/TOSC.V2022.I3.20-72
42. Hilton, P., Pedersen, J.: Catalan numbers, their generalization, and their uses. The mathematical intelligencer **13**(2), 64–75 (1991)

43. Horowitz, E., Sahni, S.: On computing the exact determinant of matrices with polynomial entries. Journal of the ACM **22**(1), 38–50 (1975). https://doi.org/10.1145/321864.321868
44. Huang, Q.: Sparse polynomial interpolation based on derivatives. J. Symb. Comput. **114**, 359–375 (2023). https://doi.org/10.1016/J.JSC.2022.06.002
45. Jinadu, A., Monagan, M.: A new dixon resultant algorithm for solving parametric polynomial systems using sparse multivariate rational function interpolation. Preprint, submitted to the Journal of Symbolic Computation, January 2025 (2025), https://www.cecm.sfu.ca/CAG/papers/AyoolaJSC25.pdf, available online
46. Jinadu, A., Monagan, M.B.: An interpolation algorithm for computing dixon resultants. In: Boulier, F., England, M., Sadykov, T.M., Vorozhtsov, E.V. (eds.) Computer Algebra in Scientific Computing - 24th International Workshop, CASC 2022, Gebze, Turkey, August 22-26, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13366, pp. 185–205. Springer (2022). https://doi.org/10.1007/978-3-031-14788-3_11
47. Jinadu, A.I.: Solving Parametric Systems Using Dixon Resultants and Sparse Interpolation Tools. Ph.D. thesis, Simon Fraser University (Summer 2023)
48. Kapur, D., Saxena, T.: Sparsity considerations in dixon resultants. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996. pp. 184–191. ACM (1996). https://doi.org/10.1145/237814.237862
49. Kapur, D., Saxena, T.: Extraneous factors in the dixon resultant formulation. In: Char, B.W., Wang, P.S., Küchlin, W. (eds.) Proceedings of the 1997 International Symposium on Symbolic and Algebraic Computation, ISSAC 1997, Maui, Hawaii, USA, July 21-23, 1997. pp. 141–148. ACM (1997). https://doi.org/10.1145/258726.258768
50. Kapur, D., Saxena, T., Yang, L.: Algebraic and geometric reasoning using dixon resultants. In: MacCallum, M.A.H. (ed.) Proceedings of the International Symposium on Symbolic and Algebraic Computation, ISSAC '94, Oxford, UK, July 20-22, 1994. pp. 99–107. ACM (1994). https://doi.org/10.1145/190347.190372
51. Koschatko, K., Lüftenegger, R., Rechberger, C.: Exploring the six worlds of gröbner basis cryptanalysis: Application to anemoi. IACR Trans. Symmetric Cryptol. **2024**(4), 138–190 (2024). https://doi.org/10.46586/TOSC.V2024.I4.138-190
52. Labahn, G., Neiger, V., Zhou, W.: Fast, deterministic computation of the hermite normal form and determinant of a polynomial matrix. J. Complex. **42**, 44–71 (2017). https://doi.org/10.1016/J.JCO.2017.03.003
53. Lewis, R.H.: Fermat computer algebra system. https://home.bway.net/lewis/, accessed: 2025-10-03
54. Lewis, R.H.: Dixon-edf: the premier method for solution of parametric polynomial systems. In: Special Sessions in Applications of Computer Algebra, pp. 237–256. Springer (2015)
55. Lewis, R.H.: Dixon-edf: The premier method for solution of parametric polynomial systems. In: Kotsireas, I.S., Martínez-Moro, E. (eds.) Applications of Computer Algebra. pp. 237–256. Springer International Publishing, Cham (2017)
56. Li, Y.: An effective algorithm of computing symbolic determinants with multivariate polynomial entries. Applied Mathematics and Computation **192**(2), 382–388 (2007)
57. Liu, F., Mahzoun, M., Meier, W.: Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, RAIN and biscuit. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International

Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 424–456. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_14
58. Macaulay, F.S.: Some formulae in elimination. Proceedings of the London Mathematical Society **1**(1), 3–27 (1902)
59. May, A., Ostuzzi, M., Ressler, H.: Just guess: Improved (quantum) algorithm for the underdetermined MQ problem. IACR Cryptol. ePrint Arch. **2025**, 1788 (2025), https://eprint.iacr.org/2025/1788
60. Meurant, G.: Hessenberg and tridiagonal matrices: Theory and examples (2025), https://epubs.siam.org/doi/10.1137/1.9781611978452
61. Painvin, L.: Note sur la méthode d'élimination de bezout. Nouvelles annales de mathématiques: journal des candidats aux écoles polytechnique et normale **13**, 278–285 (1874)
62. Qin, X., Wu, D., Tang, L., Ji, Z.: Complexity of constructing dixon resultant matrix. Int. J. Comput. Math. **94**(10), 2074–2088 (2017). https://doi.org/10.1080/00207160.2016.1276572
63. Qin, X., Zhang, L., Yang, L., Cao, S.: Heuristics to sift extraneous factors in dixon resultants. J. Symb. Comput. **112**, 105–121 (2022). https://doi.org/10.1016/J.JSC.2022.01.003
64. Roy, A., Steiner, M.J., Trevisani, S.: Arion: Arithmetization-oriented permutation and hashing from generalized triangular dynamical systems. CoRR **abs/2303.04639** (2023). https://doi.org/10.48550/ARXIV.2303.04639
65. Sauer, J.F., Szepieniec, A.: Sok: Gröbner basis algorithms for arithmetization oriented ciphers. IACR Cryptol. ePrint Arch. p. 870 (2021), https://eprint.iacr.org/2021/870
66. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM (JACM) **27**(4), 701–717 (1980)
67. Spaenlehauer, P.: Solving multi-homogeneous and determinantal systems: algorithms, complexity, applications. (Résolution de systèmes multi-homogènes et déterminantiels : algorithmes, complexité, applications). Ph.D. thesis, Pierre and Marie Curie University, Paris, France (2012), https://tel.archives-ouvertes.fr/tel-01110756
68. Steiner, M.J.: Solving degree bounds for iterated polynomial systems. IACR Trans. Symmetric Cryptol. **2024**(1), 357–411 (2024). https://doi.org/10.46586/TOSC.V2024.I1.357-411
69. Storjohann, A.: Algorithms for matrix canonical forms. Ph.D. thesis, ETH Zurich (2000)
70. Sturmfels, B.: Solving Systems of Polynomial Equations. Conference Board of the Mathematical Sciences Regional Confe, Conference Board of the Mathematical Sciences (2002), https://books.google.com.tw/books?id=WUAjMQAACAAJ
71. Szepieniec, A., Lemmens, A., Sauer, J.F., Threadbare, B., Al-Kindi: The Tip5 hash function for recursive STARKs. Cryptology ePrint Archive, Paper 2023/107 (2023), https://eprint.iacr.org/2023/107
72. Tang, X., Feng, Y.: A new efficient algorithm for solving systems of multivariate polynomial equations. Cryptology ePrint Archive, Paper 2005/312 (2005), https://eprint.iacr.org/2005/312
73. Villard, G.: On computing the resultant of generic bivariate polynomials. In: Proceedings of the 2018 International Symposium on Symbolic and Algebraic Computation, ISSAC 2018. pp. 391–398. ACM (2018). https://doi.

org/10.1145/3208976.3209020, https://perso.ens-lyon.fr/gilles.villard/BIBLIOGRAPHIE/PDF/vil18.pdf
74. Wang, D.: Solving polynomial equations: Characteristic sets and triangular systems. Mathematics and Computers in Simulation **42**(4-6), 339–351 (1996). https://doi.org/10.1016/S0378-4754(96)00008-0
75. Yang, H., Zheng, Q., Yang, J., Liu, Q., Tang, D.: A new security evaluation method based on resultant for arithmetic-oriented algorithms. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 457–489. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_15
76. Zhao, S., Fu, H.: An extended fast algorithm for constructing the dixon resultant matrix. Science in China Series A: Mathematics **48**(1), 131–143 (2005)
77. Zippel, R.: Probabilistic algorithms for sparse polynomials. In: International symposium on symbolic and algebraic manipulation. pp. 216–226. Springer (1979)

Supporting Material

Table of Contents

A Background on Algebraic Geometry and Combinatorics

In this appendix, we provide essential background on convex geometry, polynomial support structures, and lattice path enumeration that facilitate understanding of the complexity analysis in Section 3. For a detailed introduction to these topics, please refer to [26,70,19].

Definition 5 (Generic Property). *Fix degree bounds $d_1, \ldots, d_n$, and let $\mathcal{C}$ be the affine space of coefficient tuples of polynomial systems $(f_1, \ldots, f_n)$ with* $\deg(f_i) \leq d_i$. *A property $\mathcal{P}$ is said to hold* generically *if there exists a Zariski open dense subset $U \subseteq \mathcal{C}$ such that $\mathcal{P}$ holds for every $(f_1, \ldots, f_n) \in U$.*

Informally, a property holds generically if it holds for "most" polynomial systems—equivalently, for dense randomly generated polynomial systems. The set of systems for which the property fails forms a proper algebraic subvariety of lower dimension in the coefficient space.

Definition 6 (Convex Set and Convex Hull). *A set $C \subset \mathbb{R}^n$ is* convex *if for any two points $p, q \in C$, the line segment $\{(1-\lambda)p + \lambda q : \lambda \in [0,1]\}$ lies entirely in C.*

For any set $S \subset \mathbb{R}^n$, its convex hull *$conv(S)$ is the smallest convex set containing S, equivalently:*

$$conv(S) = \left\{ \sum_{i=1}^{m} \lambda_i s_i : s_i \in S, \lambda_i \geq 0, \sum_{i=1}^{m} \lambda_i = 1 \right\}.$$

Linear combinations of this form are called convex combinations.

Definition 7 (Polytope). *A* polytope *is the convex hull of a finite set in $\mathbb{R}^n$. A* lattice polytope *is a polytope whose vertices lie in $\mathbb{Z}^n$.*

Definition 8 (Monomial Support). *For a polynomial $f = \sum_\alpha c_\alpha x^\alpha$ where $c_\alpha \in \mathbb{K}$ and $\alpha \in \mathbb{Z}^n_{\geq 0}$, the* support *of f is the set of exponent vectors corresponding to nonzero coefficients:*

$$Supp(f) = \{\alpha \in \mathbb{Z}^n_{\geq 0} : c_\alpha \neq 0\}.$$

Definition 9 (Newton Polytope). *The* Newton polytope *of a polynomial f is the convex hull of its support:*

$$NP(f) = conv(Supp(f)) \subset \mathbb{R}^n.$$

The Newton polytope records the "shape" or sparsity structure of f; the actual coefficient values do not affect $NP(f)$.

Definition 10 (Mixed and Unmixed Systems [48]). *A polynomial system $\mathcal{F} = (f_1, \ldots, f_n)$ is called* unmixed *if*

$$\mathrm{NP}(f_1) = \cdots = \mathrm{NP}(f_n).$$

Otherwise, it is called mixed.

Definition 11 (Minkowski Sum). *For polytopes $P_1, P_2 \subset \mathbb{R}^n$, their Minkowski sum is:*

$$P_1 + P_2 = \{p_1 + p_2 : p_1 \in P_1, p_2 \in P_2\}.$$

More generally, for $\lambda \geq 0$ and polytope P, define $\lambda P = \{\lambda p : p \in P\}$.

Proposition 1 (Properties of Minkowski Sum).

1. *The Minkowski sum of polytopes is a polytope.*
2. *The Minkowski sum of lattice polytopes is a lattice polytope.*
3. *For polytopes P, Q and $\lambda, \mu \geq 0$: $\lambda(P + Q) = \lambda P + \lambda Q$ and $(\lambda + \mu)P = \lambda P + \mu P$.*

Proposition 2 (Newton Polytope of Products). *For polynomials $f, g \in \mathbb{K}[x_1, \ldots, x_n]$:*

$$NP(fg) = NP(f) + NP(g).$$

Proposition 3 (Faces of Minkowski Sums). *Let $P_1, \ldots, P_r \subset \mathbb{R}^n$ be polytopes, and let $P = P_1 + \cdots + P_r$ be their Minkowski sum. Then every face P' of P can be expressed as a Minkowski sum:*

$$P' = P'_1 + \cdots + P'_r,$$

where each P'_i is a face of P_i.

Definition 12 (Lattice Path). *A* lattice path *in $\mathbb{Z}^2_{\geq 0}$ from $(0,0)$ to (m,n) is a sequence of points*

$$(x_0, y_0), (x_1, y_1), \ldots, (x_k, y_k)$$

such that $(x_0, y_0) = (0,0)$, $(x_k, y_k) = (m,n)$, and for each $i = 0, 1, \ldots, k-1$ we have either

$$(x_{i+1}, y_{i+1}) = (x_i + 1, y_i) \quad \textit{(a right step)}$$

or

$$(x_{i+1}, y_{i+1}) = (x_i, y_i + 1) \quad \textit{(an up step).}$$

All points of the path lie in $\mathbb{Z}^2_{\geq 0}$. Such a path consists of exactly m right steps and n up steps. The number of lattice paths refers to the total number of distinct lattice paths satisfying these conditions.

B Proof of Lemma 2

Proof. We establish the equality by showing both inclusions.

Setup. For each $k \in \{1, \ldots, n\}$, define

$$P_k := \{(i_1, \ldots, i_k, 0, \ldots, 0) \in \mathbb{Z}^n_{\geq 0} : i_1 + \cdots + i_k \leq d_k\}.$$

By definition of the Iterated-Simplex Polynomial,

$$\mathrm{Supp}(I_{d_1,d_2,\ldots,d_n}) = P_1 + P_2 + \cdots + P_n,$$

where $+$ denotes the Minkowski sum. Define the target set:

$$Q_n := \left\{ (\alpha_1, \ldots, \alpha_n) \in \mathbb{Z}^n_{\geq 0} : \sum_{\ell=n-j+1}^{n} \alpha_\ell \leq \sum_{i=1}^{j} d_{n-i+1} \text{ for all } j \in \{1, \ldots, n\} \right\}.$$

Inclusion $P_1 + \cdots + P_n \subseteq Q_n$. Let $(\alpha_1, \ldots, \alpha_n) \in P_1 + \cdots + P_n$. Then there exist $(i_1^{(k)}, \ldots, i_n^{(k)}) \in P_k$ for $k = 1, \ldots, n$ such that

$$(\alpha_1, \ldots, \alpha_n) = \sum_{k=1}^{n} (i_1^{(k)}, \ldots, i_n^{(k)}).$$

Fix $j \in \{1, \ldots, n\}$ and consider the last j components. For each $k \in \{n-j+1, \ldots, n\}$, by the definition of P_k, we have

$$\sum_{\ell=n-j+1}^{n} i_\ell^{(k)} \leq d_k.$$

Therefore,

$$\sum_{\ell=n-j+1}^{n} \alpha_\ell = \sum_{\ell=n-j+1}^{n} \sum_{k=1}^{n} i_\ell^{(k)} = \sum_{k=n-j+1}^{n} \sum_{\ell=n-j+1}^{n} i_\ell^{(k)}$$

$$\leq \sum_{k=n-j+1}^{n} d_k = \sum_{i=1}^{j} d_{n-i+1}.$$

This shows $(\alpha_1, \ldots, \alpha_n) \in Q_n$.

Inclusion $Q_n \subseteq P_1 + \cdots + P_n$. The strategy is to construct the decomposition by greedily moving mass from the last coordinates of α into P_n (which can absorb up to d_n units of total mass), while keeping the prefix $(\alpha'_1, \ldots, \alpha'_{n-1}, 0)$ inside Q_{n-1}. We proceed by induction on n. For $n = 1$, clearly $P_1 = Q_1$.

Assume $n \geq 2$ and $P_1 + \cdots + P_{n-1} = Q_{n-1}$ (viewing Q_{n-1} as a subset of $\mathbb{Z}^{n-1}_{\geq 0} \times \{0\}$). Let $(\alpha_1, \ldots, \alpha_n) \in Q_n$. We construct a decomposition

$$(\alpha_1, \ldots, \alpha_n) = (\alpha'_1, \ldots, \alpha'_{n-1}, 0) + (\alpha''_1, \ldots, \alpha''_n)$$

with $(\alpha'_1, \dots, \alpha'_{n-1}, 0) \in Q_{n-1}$ and $(\alpha''_1, \dots, \alpha''_n) \in P_n$.

We define the decomposition recursively for $j = 1, \dots, n$. At step j, if

$$\alpha_{n-j+1} + \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell \le \sum_{i=1}^{j-1} d_{n-i+1}$$

(with the convention that the sums $\sum_{\ell=n-j+2}^{n-1} \alpha'_\ell$ and $\sum_{i=1}^{j-1} d_{n-i+1}$ are empty—and therefore equal to 0—when $j = 1$), set

$$\alpha'_{n-j+1} := \alpha_{n-j+1}, \quad \alpha''_{n-j+1} := 0.$$

Otherwise, set

$$\alpha'_{n-j+1} := \sum_{i=1}^{j-1} d_{n-i+1} - \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell, \quad \alpha''_{n-j+1} := \alpha_{n-j+1} - \alpha'_{n-j+1}.$$

In both cases,

$$\sum_{\ell=n-j+1}^{n-1} \alpha'_\ell \le \sum_{i=1}^{j-1} d_{n-i+1},$$

which establishes $(\alpha'_1, \dots, \alpha'_{n-1}, 0) \in Q_{n-1}$.

To verify $(\alpha''_1, \dots, \alpha''_n) \in P_n$, we need $\sum_{\ell=1}^{n} \alpha''_\ell \le d_n$. This is immediate in the first case. In the second case:

$$\begin{aligned}\sum_{\ell=n-j+1}^{n} \alpha''_\ell &= \alpha_{n-j+1} - \left(\sum_{i=1}^{j-1} d_{n-i+1} - \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell \right) + \sum_{\ell=n-j+2}^{n} \alpha''_\ell \\ &= \sum_{\ell=n-j+1}^{n} \alpha_\ell - \sum_{i=1}^{j-1} d_{n-i+1} \le \sum_{i=1}^{j} d_{n-i+1} - \sum_{i=1}^{j-1} d_{n-i+1} = d_n,\end{aligned}$$

where the inequality follows from the definition of Q_n.

By the induction hypothesis,

$$(\alpha_1, \dots, \alpha_n) \in Q_{n-1} + P_n \subseteq P_1 + \cdots + P_n.$$

Conclusion. Combining both inclusions, we obtain

$$\mathrm{Supp}(I_{d_1,d_2,\dots,d_n}) = P_1 + \cdots + P_n = Q_n,$$

establishing the claimed characterization.

C Experimental Validation for Dixon Matrix Size Bound

We provide numerical comparisons between our Fuss-Catalan bound (FC Bound, Corollary 1(2)), Unrestricted bound (Corollary 1(3)), the previous bound (5) from [48], and the trivial variable-based bound (4). Table 6 shows the bounds alongside the actual Dixon matrix sizes computed from randomly generated dense $(n; d)$-systems over $\mathbb{F}_p$ with $p = 2^{31} - 159$; the matrix size is field-independent for generic systems, but we record the field used for reproducibility. The "True Size" column reports the actual row/column dimension of the Dixon matrix observed in our experiments. For each pair (n, d) we sampled 20 independent, fully dense systems of n equations, all of total degree d, with all coefficients drawn uniformly and independently from $\mathbb{F}_p$; the observed size agreed across all samples. It is the support dimension $D(\mathcal{F})$, not $\operatorname{rank} \mathcal{M}$ (Remark 5).

Table 6: Comparison of Dixon Matrix Size Bounds

n	d	FC Bound	Unrestricted Bound	Bound in [48]	Variable Bound	True Size
3	2	5	6	6	8	5
3	3	12	15	13	18	12
3	4	22	28	24	32	22
3	5	35	45	37	50	35
4	2	14	20	21	48	14
4	3	55	84	72	162	55
4	4	140	220	170	384	140
4	5	285	455	333	750	285
5	2	42	70	83	384	42
5	3	273	495	421	1944	273
5	4	969	1820	1333	6144	969
5	5	2530	4845	3255	15000	2530
6	2	132	252	345	3840	132
6	3	1428	3003	2624	29160	1428
6	4	7084	15504	11059	122880	7084
6	5	23751	53130	33750	375000	23751
7	2	429	924	1493	46080	429
7	3	7752	18564	17017	524880	7752
8	2	1430	3432	6657	645120	1430
9	2	4862	12870	30368	10321920	4862
10	2	16796	48620	141093	185794560	16796

D Detailed Derivation of Complexity Ratios in Table 2

This appendix provides the step-by-step derivation for the "Ratio to Dixon" column in Table 2. All ratios below are leading-order estimates in the regime "d fixed, $n \to \infty$". The exact formulas of Table 2 are valid in every regime, but the simplified ratios are not uniform in (n, d): for n fixed and $d \to \infty$ one has instead $\binom{nd}{n} = \Theta(d^n/n!)$ and $M = C_n^{(d)} = \Theta(d^{n-1})$, and the expansions have to be redone.

For a well-determined $(n; d)$-system, we use the Dixon resultant complexity as the baseline:

$$\mathcal{T}_{\text{Dixon}} = \mathcal{O}\left(nd \cdot \left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}\right).$$

Gröbner Basis Classical [32] The complexity is:

$$\mathcal{T}_{\text{GB}} = \mathcal{O}\left(\binom{n + d_{\text{reg}}}{d_{\text{reg}}}^{\omega}\right).$$

With $d_{\text{reg}} = n(d-1)+1$, this becomes:

$$\mathcal{T}_{\text{GB}} = \mathcal{O}\left(\binom{n + n(d-1) + 1}{n(d-1)+1}^{\omega}\right) = \mathcal{O}\left(\binom{nd+1}{n}^{\omega}\right).$$

Using the binomial identity:

$$\binom{nd+1}{n} = \frac{nd+1}{nd-n+1}\binom{nd}{n},$$

we compute the ratio:

$$\begin{aligned}
\frac{\mathcal{T}_{\text{GB}}}{\mathcal{T}_{\text{Dixon}}} &= \frac{\left(\binom{nd+1}{n}\right)^{\omega}}{nd \cdot \left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} \\
&= \frac{\left(\frac{nd+1}{nd-n+1}\binom{nd}{n}\right)^{\omega}}{nd \cdot \left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} \\
&= \frac{(nd+1)^{\omega}}{nd \cdot (nd-n+1)^{\omega}} \cdot (n(d-1)+1)^{\omega} \\
&= \frac{(nd+1)^{\omega} \cdot (nd-n+1)^{\omega}}{nd \cdot (nd-n+1)^{\omega}} \\
&= \frac{(nd+1)^{\omega}}{nd} \\
&= (nd)^{\omega-1}\left(1 + \frac{1}{nd}\right)^{\omega} \\
&\sim \mathcal{O}\left((nd)^{\omega-1}\right).
\end{aligned}$$

Gröbner Basis F5 Bound [9,13] The complexity is:

$$\mathcal{T}_{\text{F5}} = \mathcal{O}\left(n \cdot d_{\text{reg}} \cdot \binom{n + d_{\text{reg}} - 1}{d_{\text{reg}}}^{\omega}\right).$$

With $d_{\text{reg}} = n(d-1)+1$, this becomes:

$$\begin{aligned}\mathcal{T}_{\text{F5}} &= \mathcal{O}\left(n \cdot (n(d-1)+1) \cdot \binom{n + n(d-1) + 1 - 1}{n(d-1)+1}^{\omega}\right) \\ &= \mathcal{O}\left(n \cdot (n(d-1)+1) \cdot \binom{nd}{n-1}^{\omega}\right).\end{aligned}$$

Using the identity:

$$\binom{nd}{n-1} = \frac{n}{nd-n+1}\binom{nd}{n},$$

the ratio becomes:

$$\begin{aligned}\frac{\mathcal{T}_{\text{F5}}}{\mathcal{T}_{\text{Dixon}}} &= \frac{n(nd-n+1)\left(\frac{n}{nd-n+1}\binom{nd}{n}\right)^{\omega}}{nd\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} \\ &= \frac{n(nd-n+1) \cdot n^{\omega} \cdot (n(d-1)+1)^{\omega}}{nd \cdot (nd-n+1)^{\omega}} \\ &= \frac{n^{\omega+1} \cdot (nd-n+1)}{nd} \\ &= n^{\omega+1}\left(1 - \frac{1}{d} + \frac{1}{nd}\right) \\ &\sim \mathcal{O}\left(n^{\omega+1}\right).\end{aligned}$$

Gröbner Basis F5 Refined [9] We first describe the origin of the corresponding complexity bound, which is obtained from the analysis of Macaulay matrices combined with fast linear algebra [9, section 1.3]. For a well-determined system of n equations in n variables of degree d, the Macaulay matrix at degree $d_{\text{reg}} = n(d-1)+1$ has $\binom{n+d_{\text{reg}}}{n}$ columns (indexed by monomials of degree at most d_{reg}) and $n\binom{n+d_{\text{reg}}-d}{n}$ rows (indexed by multipliers $x^{\alpha} f_i$ with $\deg(x^{\alpha}) \leq d_{\text{reg}} - d$). Its rank is $\binom{n+d_{\text{reg}}}{n} - d^n$, where the lower-order term d^n can be neglected in the complexity estimate. Using the fact that the row echelon form of a matrix with r rows, c columns and rank ρ can be computed in time $O(rc\rho^{\omega-2})$ [69], we obtain the bound shown in Table 2.

The complexity after substituting d_{reg} becomes:

$$\mathcal{T}_{\text{F5P}} = \mathcal{O}\left(n\binom{d(n-1)+1}{n}\binom{nd+1}{n}^{\omega-1}\right).$$

Substituting the binomial identity:

$$\frac{\mathcal{T}_{\text{F5P}}}{\mathcal{T}_{\text{Dixon}}} = \frac{n\binom{nd-d+1}{n}\left(\frac{nd+1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega-1}}{nd\cdot\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} = \frac{n\binom{nd-d+1}{n}}{nd\binom{nd}{n}}\cdot(nd-n+1)\cdot(nd+1)^{\omega-1}.$$

Using the asymptotic estimate

$$\frac{\binom{nd-d+1}{n}}{\binom{nd}{n}} = \prod_{i=0}^{n-1}\frac{nd-d+1-i}{nd-i} = \prod_{i=0}^{n-1}\left(1-\frac{d-1}{nd-i}\right) \approx \left(\frac{d-1}{d}\right)^{d-1},$$

where the last step uses $\sum_{i=0}^{n-1}\ln(1-(d-1)/(nd-i)) \approx -(d-1)(H_{nd}-H_{nd-n}) \approx -(d-1)\ln(d/(d-1))$ (with H_m the m-th harmonic number), we obtain:

$$\frac{\mathcal{T}_{\text{F5P}}}{\mathcal{T}_{\text{Dixon}}} \approx \left(\frac{d-1}{d}\right)^{d-1}\cdot\frac{nd-n+1}{d}\cdot(nd+1)^{\omega-1} \sim \mathcal{O}\left(n^{\omega}d^{\omega-1}\right),$$

where the d-dependent constant $((d-1)/d)^{d-1}$ is absorbed into the big-$\mathcal{O}$. The limiting value $\lim_{d\to\infty}((d-1)/d)^{d-1} = e^{-1}$ is approached only as $d\to\infty$.

FGLM [33] The complexity is:

$$\mathcal{T}_{\text{FGLM}} = \mathcal{O}(n\cdot d_I^{\omega}) = \mathcal{O}(n\cdot(d^n)^{\omega}) = \mathcal{O}(n\cdot d^{n\omega}).$$

Using Stirling's approximation (valid for d fixed and $n\to\infty$):

$$\binom{nd}{n} \approx \frac{1}{\sqrt{2\pi n}}\sqrt{\frac{d}{d-1}}\left(\frac{d^d}{(d-1)^{d-1}}\right)^n,$$

the ratio is:

$$\begin{aligned}\frac{\mathcal{T}_{\text{FGLM}}}{\mathcal{T}_{\text{Dixon}}} &= \frac{n\cdot d^{n\omega}}{nd\cdot\left(\frac{1}{n(d-1)+1}\cdot\frac{1}{\sqrt{2\pi n}}\left(\frac{d^d}{(d-1)^{d-1}}\right)^n\right)^{\omega}}\\ &\propto \text{poly}(n,d)\cdot\left(\frac{d^{n\omega}}{\left(\frac{d^d}{(d-1)^{d-1}}\right)^{n\omega}}\right)\\ &= \text{poly}(n,d)\cdot\left(\frac{(d-1)^{d-1}}{d^{d-1}}\right)^{n\omega}\\ &= \text{poly}(n,d)\cdot\left(\left(1-\frac{1}{d}\right)^{d-1}\right)^{n\omega}.\end{aligned}$$

Since:

$$\lim_{d\to\infty}\left(1-\frac{1}{d}\right)^{d-1}=e^{-1},$$

the ratio is:

$$\frac{\mathcal{T}_{\mathrm{FGLM}}}{\mathcal{T}_{\mathrm{Dixon}}}\sim\mathcal{O}\left(\left(1-\frac{1}{d}\right)^{(d-1)n\omega}\right)\xrightarrow{d\to\infty}\mathcal{O}\left(e^{-n\omega}\right),$$

which shows an exponential decrease as n increases for every fixed $d>1$. The further limit $d\to\infty$ gives the exponential factor $e^{-n\omega}$; for finite d (which is the practical regime for AO primitives), the base is $((d-1)/d)^{d-1}>e^{-1}$.

Other elimination and change-of-order algorithms. The FGLM estimate $\mathcal{O}(n\,d_{\mathcal{I}}^{\omega})$ used above does not include the construction of the multiplication matrices, and it can be improved under shape and stability assumptions by the HNF-based change of order of Berthomieu et al. [15]. For two eliminated variables, structured Sylvester-resultant algorithms such as Villard's [73] have a better asymptotic complexity, even though the Dixon construction works with a smaller matrix. On the AO systems of Section 5 the relevant Gröbner competitors are moreover the FreeLunch [12] and CheapLunch [7] attacks, for which the grevlex basis is essentially free and the change-of-order step dominates; Dixon is then attractive in the few-variable/high-degree regime, while CheapLunch is expected to be preferable as the number of variables grows.

E Further Comparative Plots

We present a more comprehensive comparison of the computational complexity of the Dixon resultant and Gröbner basis methods from multiple perspectives.

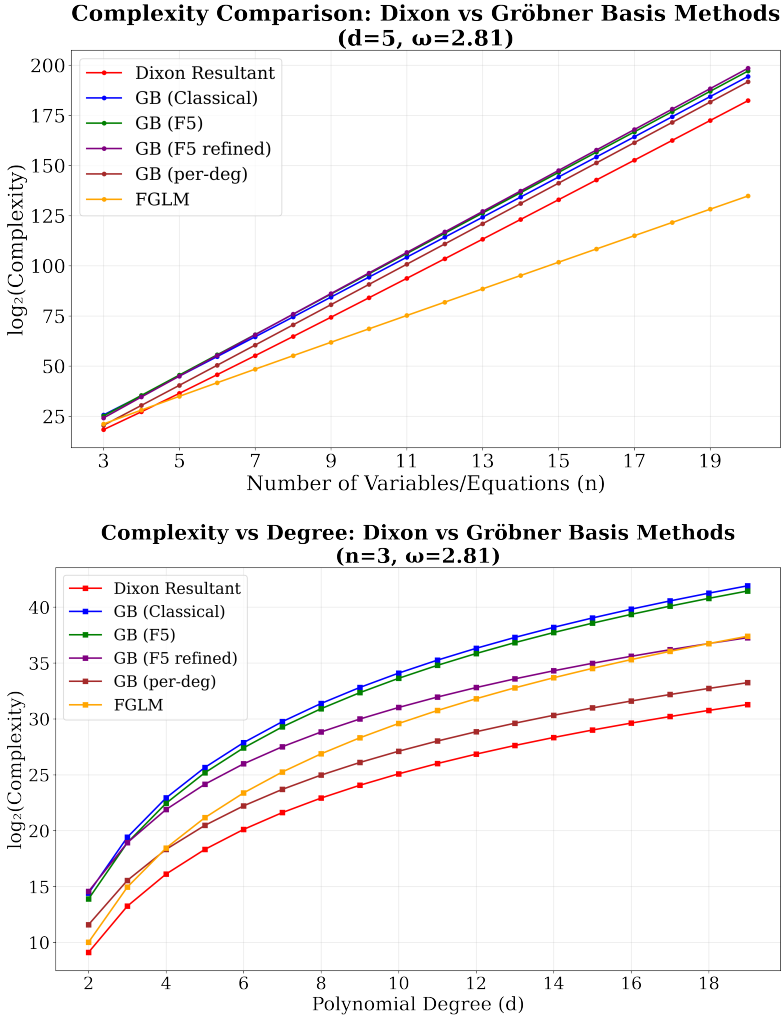

Fig. 6: Complexity comparison between the Dixon resultant and Gröbner basis

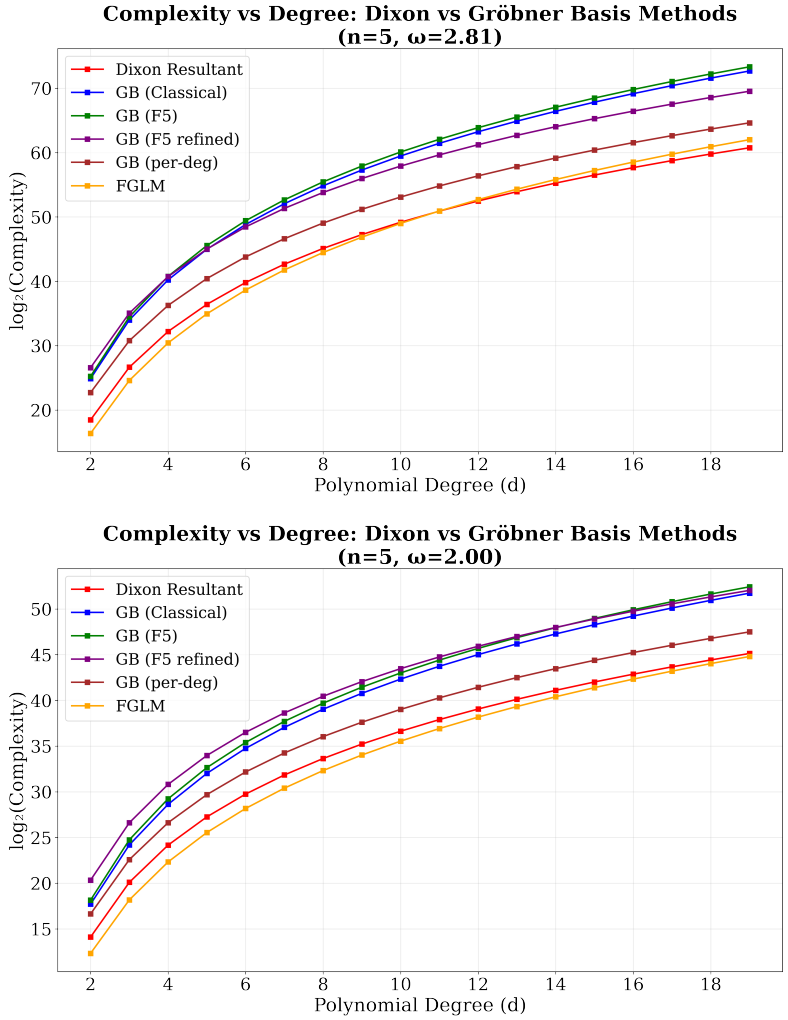

Fig. 6: Complexity comparison between the Dixon resultant and Gröbner basis (continued).

F Degree-Aware Submatrix Selection Algorithms

Algorithm 1 gives the pseudocode for the selection strategy described in Section 4.1.

Algorithm 1: Degree-aware maximal-rank submatrix selection

Input: Polynomial matrix $M \in \mathbb{K}[t_1, \ldots, t_m]^{r \times c}$ (possibly rectangular or rank-deficient)
Output: Row indices $\mathcal{R}$ and column indices $\mathcal{C}$ with $|\mathcal{R}| = |\mathcal{C}|$ such that $M[\mathcal{R}, \mathcal{C}]$ has maximal rank and minimal degree estimate
Initialize $\mathcal{R} \leftarrow \emptyset$, $\mathcal{C} \leftarrow \emptyset$
Choose concrete evaluation point $\alpha = (\alpha_1, \ldots, \alpha_m) \in \mathbb{F}^m$
Evaluate $M_{\text{const}} \leftarrow M(\alpha_1, \ldots, \alpha_m)$ {Constant matrix}
Compute row degree weights: $d_r(i) \leftarrow \max_j \deg(M_{i,j})$ for each row i
Compute column degree weights: $d_c(j) \leftarrow \max_i \deg(M_{i,j})$ for each column j
Sort row indices by increasing degree weights: ρ_r
Sort column indices by increasing degree weights: ρ_c
Initialize basis tracker: $\mathcal{B} \leftarrow \emptyset$
for each candidate index k in ρ_r **do**
    Extract row vector $\boldsymbol{v}_k$ from M_{const}
    Eliminate $\boldsymbol{v}_k$ against existing basis vectors in $\mathcal{B}$
    if $\boldsymbol{v}_k \neq \mathbf{0}$ after elimination **then**
        Normalize $\boldsymbol{v}_k$ and add it to $\mathcal{B}$
        Include k in $\mathcal{R}$
    end if
end for
Reset basis tracker: $\mathcal{B} \leftarrow \emptyset$
for each candidate index k in ρ_c **do**
    Extract column vector $\boldsymbol{v}_k$ from M_{const}
    Eliminate $\boldsymbol{v}_k$ against existing basis vectors in $\mathcal{B}$
    if $\boldsymbol{v}_k \neq \mathbf{0}$ after elimination **then**
        Normalize $\boldsymbol{v}_k$ and add it to $\mathcal{B}$
        Include k in $\mathcal{C}$
    end if
end for
return $\mathcal{R}, \mathcal{C}$

It is straightforward to observe that the computational cost of our algorithm essentially reduces to two Gaussian eliminations on constant matrices: one for row selection and the other for column selection. Compared with the complexity in Equation (13), this step incurs no additional asymptotic overhead.

G Experimental Results of Degree-Aware Submatrix Selection

We present the experimental results of Algorithm 1 to validate Heuristic 2. Table 7 compares the performance of our degree-aware selection strategy against naive random selection across various polynomial system configurations.

Table 7: Comparison of Extraneous Factor Occurrence

n_x	k	Degrees $(d_1, \ldots, d_{n_x+1})$	Bézout Bound $\prod d_i$	Extraneous Factors	
				Naive	Algorithm 1
2	1	$(3, 4, 5)$	60	195/200	0/200
2	1	$(5, 6, 6)$	180	200/200	0/200
2	1	$(5, 5, 5)$	125	192/200	0/200
5	1	$(2, 2, 2, 2, 2, 2)$	64	199/200	0/200
3	2	$(2, 2, 3, 3)$	36	177/200	1/200
Extraneous Factor Probability				963/1000	1/1000

In Table 7, n_x denotes the number of variables to eliminate (so the system contains $n_x + 1$ polynomials, consistent with the convention in the main text where n is the number of polynomials), k denotes the number of parameters retained. Extraneous factors occur when $\deg(\mathrm{Res}(\mathcal{F})) > \prod d_i$ (Bézout bound violation). The naive method uses random submatrix selection. All experiments were conducted over $\mathbb{F}_{65537}$ with 20 repetitions per system configuration. In the few configurations where Algorithm 1 did not fully remove the extraneous factors, the resulting resultant degree exceeded the Bézout bound by at most 1–2, so the impact on the overall complexity estimates is negligible.

H Detailed Stage-wise Complexity Parameters

This appendix provides the detailed parameter derivations and the full set of five determinant models (expansion, Bareiss, Kronecker+HNF, interpolation, sparse) for the two symbolic-determinant stages whose preferred models were summarized in Section 3.2. We keep the notation of Section 3.2 throughout: $\mathcal{F}$ is an $(n; d)$-system in $n-1+k$ variables, $M = C_n^{(d)}$ is the generic Dixon-matrix size, and we use the determinant-method complexity templates from Appendix I.

H.1 Step 1 Parameter Derivations

This subsection derives the four abstract parameters (μ_1, S_1, L_1, K_1) and the additional grid-size parameter N_1 used to instantiate the five Step 1 complexity models. The expressions for (μ_1, S_1, L_1) recover (11) in Section 3.2, and substituting them into the determinant templates of Table 1 reproduces (11)–(12).

The refined construction of the cancellation matrix from (3) lowers several *dual*-variable degrees by one through the difference quotients in $x_i - \bar{x}_i$, but this affects only the eliminated and dual variables and not the retained parameters $\boldsymbol{a}$. For the asymptotic estimates below we use uniform upper bounds that are simpler to state. The determinant in Step 1 involves $\ell = 2n+k-2$ variables (the $n-1$ original variables, the $n-1$ dual variables, and the k retained parameters), with each entry having parameter total degree at most d. Summing across the n rows of the determinant, the parameter total degree of $\det(\widehat{C})$ is therefore at most

$$B_1 := \sum_{i=1}^{n} d = nd.$$

Dense monomial-count proxy. The expansion-by-minors model uses the dense monomial-count proxy

$$\mu_1 \le \binom{n-1+k+d}{n-1+k},$$

i.e. the number of dense monomials in the original f_j when viewed as a polynomial in the $n-1+k$ variables of $\boldsymbol{x}$ and $\boldsymbol{a}$ of total degree at most d. We use this as a per-entry dense bound; intermediate minors arising in the expansion are also bounded by S_1 (see below).

Variable-wise degree bounds. The Kronecker+HNF model uses the variable-wise degree bounds derived from the refined cancellation-matrix structure. A row-by-row analysis of $\widehat{C}$ (the variable x_i appears in rows $1, \dots, i$ with degree $\le d$, in row $i+1$ with degree $\le d-1$ after the difference quotient, and is absent from rows $i+2, \dots, n$ where it has been replaced by $\bar{x}_i$; an analogous count applies to $\bar{x}_i$) gives

$$b_{x_i} := \deg_{x_i}(\det \widehat{C}) \le (i+1)d - 1, \qquad b_{\bar{x}_i} := \deg_{\bar{x}_i}(\det \widehat{C}) \le (n-i)d - 1,$$
$$b_{a_j} := \deg_{a_j}(\det \widehat{C}) \le nd,$$

for $i = 1, \ldots, n-1$ and $j = 1, \ldots, k$. Per-entry degree bounds are $e_{x_i}, e_{\bar{x}_i}, e_{a_j} \le d$. Kronecker substitution must use the determinant-level base $b_* + 1$ (per Appendix I). For concreteness, let $b_1, \ldots, b_\ell$ denote the variable-wise determinant degrees in some fixed ordering of the $\ell = 2n + k - 2$ variables, with the last variable playing the role of the univariate indeterminate t after Kronecker substitution. After substitution, the average column degree of the univariate matrix is bounded by

$$K_1 := d \cdot \prod_{i=1}^{\ell-1}(b_i + 1) \;\le\; \prod_{i=1}^{\ell}(b_i + 1),$$

where the first factor d is the per-entry degree bound of the indeterminate variable. The bound is invariant under the choice of which variable serves as t. The HNF cost is then $T_1^{\mathrm{HNF}} = \widetilde{O}(n^\omega K_1)$.

Interpolation parameters. For evaluation-interpolation, the tensor-grid size is

$$N_1 = \prod_{i=1}^{2n+k-2} (b_i + 1),$$

and one probe evaluates the n^2 entries of $\widehat{C}$ at a point of $\mathbb{F}_q^\ell$ and computes the determinant of the resulting $n \times n$ constant matrix. Each entry f_j is a polynomial in the $n - 1 + k$ variables of $\boldsymbol{x}, \boldsymbol{a}$ with at most μ_1 monomials, so building the evaluated matrix from n underlying polynomials at n points costs $\widetilde{O}(n^2\mu_1)$ field operations, and the constant determinant costs $\widetilde{O}(n^\omega)$. Therefore

$$L_1 = \widetilde{O}(n^\omega + n^2\mu_1).$$

Sparse term bound. The sparse model uses the structural term bound

$$S_1 \le M^2 \binom{B_1 + k}{k},$$

where the factor M^2 comes from the bilinear Dixon support (each of $\boldsymbol{x}, \bar{\boldsymbol{x}}$ contributes at most M distinct monomials by Lemma 4), and $\binom{B_1+k}{k}$ bounds the number of parameter monomials of total degree at most B_1 in the k retained parameters $\boldsymbol{a}$; see Appendix I for the full derivation.

Step 1 complexity models. Substituting these parameters into the determinant models of Table 1 yields the following five Step 1 complexity estimates (all in field operations over $\mathbb{F}_q$):

$$\begin{aligned}
T_1^{\mathrm{minor}} &= \widetilde{O}(n 2^n \mu_1 S_1), \\
T_1^{\mathrm{Bareiss}} &= \widetilde{O}(n^3 S_1^2), \\
T_1^{\mathrm{HNF}} &= \widetilde{O}(n^\omega K_1), \\
T_1^{\mathrm{eval}} &= \widetilde{O}(N_1(L_1 + \ell B_1)), \\
T_1^{\mathrm{sparse}} &= \widetilde{O}(L_1 S_1 + S_1 \log q).
\end{aligned}$$

In the expansion and Bareiss models we use S_1 as the support proxy U for intermediate minors, justified by the genericity argument of Appendix I.

FDixon recursive construction. Besides the five determinant models, the direct recursive construction of Zhao and Fu [76] builds the Dixon matrix *without* first forming the $n \times n$ cancellation determinant, so it replaces Steps 1 and 2 by a single block-recursive construction. Following the complexity model of Qin et al. [62, Theorem 3.1], our implementation uses the surrogate

$$T_2^{\mathrm{FDixon}} = \widetilde{O}\left(m_1^2 \, (n!)^3 \prod_{i=2}^{n} m_i^3 \right),$$

where the sorted surrogate degrees m_i are the per-variable degree bounds selected in the code. (One can verify the $(n!)^3$ factor by combining the dominant term $n^3 \cdot ((n-1)!)^3$ of [62, Eq. (19)–(20)] with $n \cdot (n-1)! = n!$.) The factorial factor reflects the recursive enumeration of monomial supports inside the block recursion, which makes the estimate most attractive when n is small and the degrees m_i are large; for systems with many eliminated variables it becomes less competitive than the direct Step 1 path.

H.2 Step 4 Parameter Derivations

This subsection derives the abstract Step 4 parameters $(D_4, N_4, L_4, \delta_4, \mu_4, S_4)$ used to instantiate the five Step 4 complexity models. The expressions for $(D_4, N_4, L_4, \delta_4)$ recover (14) in Section 3.2, and substituting them into the determinant templates of Table 1 reproduces (14)–(15).

Step 4 computes a symbolic determinant of an $M \times M$ polynomial matrix in the k retained parameters $\boldsymbol{a}$. Let $E_4 \le nd$ be an entry-wise parameter-degree bound for the Step 4 matrix and $D_4 \le d^n$ be the Bézout total-degree bound for the elimination polynomial.

Parameter expressions. Kronecker substitution must use a base $\ge ME_4 + 1$ to avoid overflow in the encoded determinant; the resulting univariate matrix has average column degree bounded by

$$\delta_4 := E_4 \prod_{j=1}^{k-1} (ME_4 + 1),$$

which collapses to $\delta_4 = E_4 \le nd$ in the well-determined case $k = 1$. The evaluation-interpolation and sparse-interpolation parameters are

$$N_4 \le (D_4 + 1)^k, \qquad S_4 \le \binom{D_4 + k}{k}, \qquad L_4 = \widetilde{O}(M^\omega).$$

The dense monomial-count proxy for one matrix entry is

$$\mu_4 \le \binom{k + E_4}{k}.$$

Step 4 complexity models. The full set of five determinant models (all in field operations over $\mathbb{F}_q$) is:

$$\begin{aligned} T_4^{\text{minor}} &= \widetilde{O}(M2^M \mu_4 S_4), \\ T_4^{\text{Bareiss}} &= \widetilde{O}(M^3 S_4^2), \\ T_4^{\text{HNF}} &= \widetilde{O}(M^\omega \delta_4), \\ T_4^{\text{eval}} &= \widetilde{O}(N_4(L_4 + kD_4)), \\ T_4^{\text{sparse}} &= \widetilde{O}(L_4 S_4 + S_4 \log q). \end{aligned}$$

Full table of stage-wise models. Table 8 collects all the formulas above.

Table 8: Complexity models instantiated for the first and final symbolic determinants.

Method	**Step 1/2**	**Step 4**
Expansion	$\widetilde{O}(n2^n \mu_1 S_1)$	$\widetilde{O}(M2^M \mu_4 S_4)$
Bareiss	$\widetilde{O}(n^3 S_1^2)$	$\widetilde{O}(M^3 S_4^2)$
Kronecker	$\widetilde{O}(n^\omega K_1)$	$\widetilde{O}(M^\omega \delta_4)$
Interpolation	$\widetilde{O}(N_1(L_1 + \ell B_1))$	$\widetilde{O}(N_4(L_4 + kD_4))$
Sparse	$\widetilde{O}(L_1 S_1 + S_1 \log q)$	$\widetilde{O}(L_4 S_4 + S_4 \log q)$
FDixon	$\widetilde{O}(m_1^2 (n!)^3 \prod_{i=2}^n m_i^3)$	—

H.3 Step 1 vs Step 4 Comparison

This subsection gives the detailed comparison behind the main-text statement that the symbolic cost is usually governed by the final determinant computation. We first compare the most favorable first-determinant estimate with the univariate HNF model for the final determinant, and then relate this theoretical picture to the trend plots used in the paper.

Notation. We keep the notation of Section 3.2 and the preceding subsections. In particular, the first determinant is modeled by

$$T_1^{\text{minor}}, \quad T_1^{\text{Bareiss}}, \quad T_1^{\text{HNF}}, \quad T_1^{\text{eval}}, \quad T_1^{\text{sparse}}, \quad T_2^{\text{FDixon}},$$

while the final determinant is modeled by

$$T_4^{\text{minor}}, \quad T_4^{\text{Bareiss}}, \quad T_4^{\text{HNF}}, \quad T_4^{\text{eval}}, \quad T_4^{\text{sparse}}.$$

Comparison of T_1^{sparse} and T_4^{HNF} as n grows. Assume the well-determined setting $k = 1$ and keep d fixed. Then

$$T_1^{\mathrm{sparse}} = \widetilde{O}(L_1 S_1 + S_1 \log q) = \widetilde{O}(M^2 \operatorname{poly}(n)),$$

because $S_1 \le M^2(B_1 + 1) = \widetilde{O}(M^2)$ for $k = 1$, and one probe costs $L_1 = \widetilde{O}(n^\omega + n^2\mu_1)$ with $\mu_1 \le \binom{n+d}{n}$, hence $L_1 = \widetilde{O}(\operatorname{poly}(n))$ for fixed d (the $S_1 \log q$ term is also absorbed by $\widetilde{O}(M^2 \operatorname{poly}(n))$ since $\log q$ is polylogarithmic in the problem size). On the other hand,

$$T_4^{\mathrm{HNF}} = \widetilde{O}(nd\, M^\omega).$$

For fixed d, the Fuss–Catalan quantity satisfies $M = C_n^{(d)} = \Theta(\gamma_d^n / n^{3/2})$ for a constant $\gamma_d > 1$. Hence T_1^{sparse} grows essentially like M^2 times a polynomial factor, while T_4^{HNF} grows like M^ω times a polynomial factor. For $\omega > 2$ (which is the regime of all currently known fast linear-algebra algorithms, including the practically achievable $\omega \approx 2.81$ used in our timing tables), the dominant M-exponent of the final determinant is strictly larger, so the growth of T_4^{HNF} is asymptotically faster than that of T_1^{sparse} as n increases. The borderline case $\omega = 2$ makes the two M-exponents equal, so the two stages are of the same asymptotic order in M and their relative dominance is determined by the polynomial-in-(n, d) prefactors and implementation constants rather than by the asymptotics; this is precisely why we distinguish the two regimes.

Comparison of T_1^{HNF} and T_4^{HNF} as d grows. Fix n and again take $k = 1$. The variable-wise determinant degrees in the first determinant satisfy $b_i = \Theta(nd)$, with $\ell = 2n - 1$, and per-entry degree $\Theta(d)$, so (treating n as fixed and letting $d \to \infty$)

$$K_1 = d \cdot \prod_{i=1}^{\ell-1} (b_i + 1) = \Theta(d^{2n-1}), \qquad T_1^{\mathrm{HNF}} = \widetilde{O}(n^\omega d^{2n-1}).$$

For the final determinant,

$$M = C_n^{(d)} = \Theta(d^{n-1}), \qquad T_4^{\mathrm{HNF}} = \widetilde{O}(nd\, M^\omega) = \widetilde{O}(d^{\omega(n-1)+1}).$$

Since $\omega > 2$, we have $\omega(n-1) + 1 > 2n - 1$ for every $n \ge 2$, so T_4^{HNF} grows strictly faster than T_1^{HNF} as d increases. For the borderline case $\omega = 2$, the two exponents coincide ($\omega(n-1) + 1 = 2n - 1$), and the two stages have the same asymptotic rate in d; here the dominance depends only on the polynomial prefactors, and in fact $T_1^{\mathrm{HNF}} = \widetilde{O}(n^2 d^{2n-1})$ carries an extra factor $n^{\omega-1}$ relative to $T_4^{\mathrm{HNF}} = \widetilde{O}(n\, d^{2n-1})$, so at $\omega = 2$ the absolute ordering of the two stages is not fixed by the asymptotics. The strict, regime-independent dominance of Step 4 is therefore an $\omega > 2$ phenomenon, which is the practically relevant case: it is driven by the larger M-exponent (M being polynomially large in both n and d), and it is consistent with the measured running times, where the final symbolic determinant is the bottleneck. We accordingly state the dominance for $\omega > 2$ and treat $\omega = 2$ as an idealized boundary used only in the uniform-exponent security plots.

Consequences for the overall comparison. The two formula comparisons above explain the qualitative picture observed in the implementation and in the plots:

1. for low-variable/high-degree systems, the final determinant is clearly the dominant symbolic cost;
2. for high-variable/low-degree systems, the first determinant rises sharply, and sparse interpolation is the only first-determinant model that can stay below the final determinant;
3. the numerical submatrix-selection phase remains below the symbolic final determinant throughout the parameter ranges considered in the paper.

With this method selection the first determinant never dominates: Kronecker+HNF controls it in the high-degree regime and cached expansion or sparse interpolation controls it in the high-variable regime. This matches the measured runtimes, where the final symbolic determinant (Step 4) consumes the overwhelming majority of the time. This is why the main text uses T_{34}^{best} as the overall symbolic term. The specialization to the well-determined case $k = 1$, including the comparison of T_4^{HNF} with both T_1^{sparse} and T_1^{minor} used in Section 3.2, is treated separately in Appendix H.4.

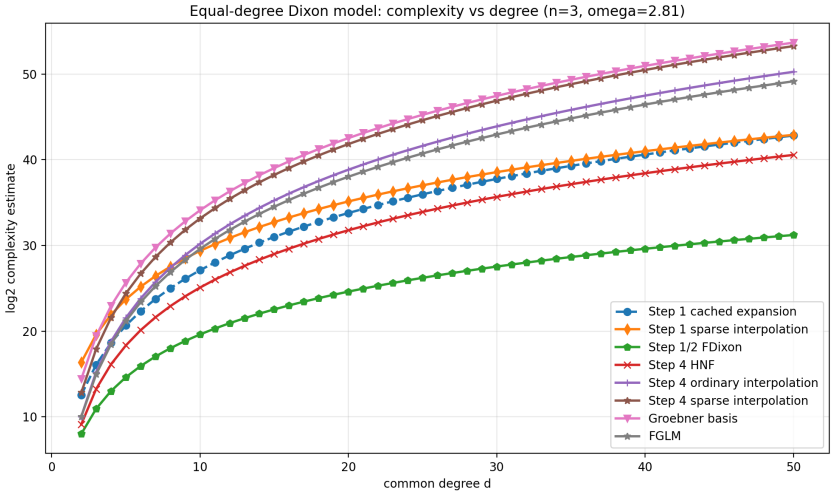

Fig. 7: Comparison of the first and final determinant costs as the polynomial degree varies. The final determinant keeps the largest growth rate in the low-variable/high-degree regime.

H.4 Well-determined Case ($k = 1$)

We now specialize the comparison in Appendix H.3 to the well-determined setting $k = 1$, which is the case used both in the comparison with Gröbner ba-

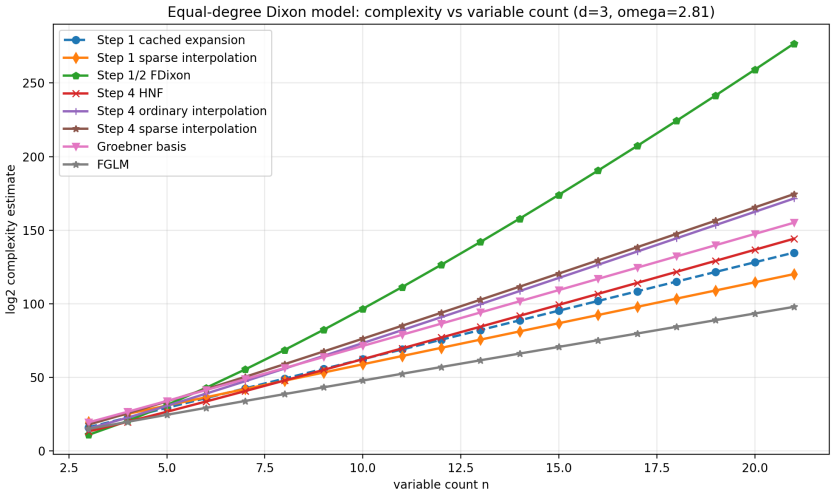

Fig. 8: Comparison of the first and final determinant costs as the number of variables varies. In the low-degree/high-variable regime, sparse interpolation is the only first-determinant model that typically remains below the final determinant.

sis methods (Section 3.3) and in the AO applications of Section 5. The argument is more explicit than the general case because the parameter space is one-dimensional, so $D_4 \le d^n$, $E_4 \le nd$, $\delta_4 = nd$, and Kronecker+HNF reduces to a single univariate HNF on an $M \times M$ matrix.

Explicit formulas. Substituting $k = 1$ into the Step 1 and Step 4 templates of Appendices H.1 and H.2 gives

$$T_4^{\mathrm{HNF}} = \widetilde{O}\big(nd\, M^{\omega}\big), \tag{31}$$

$$T_1^{\mathrm{sparse}} = \widetilde{O}\Big(n^2\,(nd+1)\binom{n+d}{n} M^2\Big) = \widetilde{O}\Big(n^3\, d\,\binom{n+d}{n} M^2\Big), \tag{32}$$

$$T_1^{\mathrm{minor}} = \widetilde{O}\Big(n\,2^n (nd+1)\binom{n+d}{n} M^2\Big) = \widetilde{O}\Big(n^2 d\,2^n \binom{n+d}{n} M^2\Big), \tag{33}$$

where, in (32), the factor $nd+1$ is combined with the n^2 probe-cost factor into the prefactor $n^3 d$ (this is the form used in the main text); the comparisons below are in any case insensitive to this d-factor, since it cancels against the nd factor of (31) when the ratio is formed.

Asymptotic size of M. For fixed $d \ge 2$ and $n \to \infty$, the Fuss–Catalan formula and Stirling's approximation give

$$M \;=\; C_n^{(d)} \;=\; \frac{1}{n(d-1)+1}\binom{nd}{n} \;=\; \Theta\Big(\frac{\gamma_d^{\,n}}{n^{3/2}}\Big), \qquad \gamma_d := \frac{d^d}{(d-1)^{d-1}}, \tag{34}$$

where $\gamma_d > 1$ for $d \geq 2$. The sequence $(\gamma_d)_{d\geq 2}$ is strictly increasing, with minimum $\gamma_2 = 4$, and grows like $\gamma_d \sim e\,d$ as $d \to \infty$. Equation (34) shows that M grows exponentially in n for any fixed $d \geq 2$, while $\binom{n+d}{n} = \Theta(n^d)$ is only polynomial in n for fixed d. This separation is the key quantitative input for the two comparisons below.

Comparison of T_4^{HNF} with T_1^{sparse}. Forming the ratio and cancelling the common d-factors gives

$$\frac{T_4^{\mathrm{HNF}}}{T_1^{\mathrm{sparse}}} \;=\; \widetilde{\Theta}\left(\frac{M^{\omega-2}}{n^2\binom{n+d}{n}}\right). \tag{35}$$

Substituting (34), the numerator $M^{\omega-2}$ equals $\gamma_d^{(\omega-2)n}/n^{3(\omega-2)/2}$, which grows *exponentially* in n as long as $\gamma_d^{\omega-2} > 1$, i.e. as long as $\omega > 2$ (since $\gamma_d > 1$ for $d \geq 2$). The denominator is polynomial in n for fixed d. Hence for every $\omega > 2$ and every fixed $d \geq 2$, the ratio (35) tends to infinity as $n \to \infty$, so

$$T_4^{\mathrm{HNF}} \;\gg\; T_1^{\mathrm{sparse}} \quad \text{asymptotically in } n.$$

Comparison of T_4^{HNF} with T_1^{minor}. The same cancellation yields

$$\frac{T_4^{\mathrm{HNF}}}{T_1^{\mathrm{minor}}} \;=\; \widetilde{\Theta}\left(\frac{M^{\omega-2}}{n\,2^n\binom{n+d}{n}}\right) \;=\; \widetilde{\Theta}\left(\frac{(\gamma_d^{\omega-2}/2)^n}{\mathrm{poly}(n,d)}\right). \tag{36}$$

This ratio tends to infinity as $n \to \infty$ *iff* the base satisfies

$$\gamma_d^{\omega-2} > 2, \qquad \text{equivalently} \qquad \omega \;>\; 2 + \frac{\log 2}{\log \gamma_d}. \tag{37}$$

The right-hand side of (37) is strictly decreasing in d: for $d = 2$ it equals $2 + \frac{\log 2}{\log 4} = 2.5$; for $d = 3$ it equals $2 + \frac{\log 2}{\log(27/4)} \approx 2.363$; and it tends to 2 from above as $d \to \infty$. In particular, for the Strassen exponent $\omega \approx 2.81$ used in our timing tables and Sections 5–3.3, condition (37) is met for all $d \geq 2$ (e.g. $\gamma_2^{0.81} \approx 3.09 > 2$). Hence

$$T_4^{\mathrm{HNF}} \;\gg\; T_1^{\mathrm{minor}} \quad \text{asymptotically in } n,$$

for every $\omega \geq 2.81$ and every $d \geq 2$ (and more generally whenever (37) holds).

Boundary case $\omega = 2$. At the idealized boundary $\omega = 2$, the M-exponents of T_4^{HNF} and of both T_1^{sparse} and T_1^{minor} coincide, so the asymptotic dominance is determined by the polynomial-in-(n,d) prefactors rather than by an M-exponent gap. The $\omega = 2$ value is used in this paper only in the uniform-exponent security plots (Section 3.3 and Figure 1), where it is applied identically to Dixon and to every Gröbner-basis variant; relative comparisons there should therefore be read as "uniform-ω ratios" rather than literal asymptotic statements about any single method. For any $\omega > 2$ already at infinitesimal margin, the $M^{\omega-2}$ factor reactivates and T_4^{HNF} dominates T_1^{sparse}; condition (37) then governs the comparison with T_1^{minor}. The Strassen exponent $\omega \approx 2.81$ used in the timing tables comfortably sits in this regime.

Consequence. Combining the two comparisons above, for all parameter ranges considered in the experiments (Strassen $\omega \approx 2.81$, $d \geq 2$, and n moderate to large), neither T_1^{sparse} nor T_1^{minor} exceeds T_4^{HNF}, and the measured running times confirm that Step 4 is the dominant phase. We therefore use $T = T_4^{\text{HNF}}$, as in (16), as the overall complexity estimate for well-determined systems.

I Alternative Determinant Computation Methods

This appendix expands the determinant-method summary from Table 1. We record both the complexity formulas and the stage-specific origin of the parameters used in the comparison code.

Notation. Let $P(\boldsymbol{y})$ be an $s \times s$ polynomial matrix in variables $\boldsymbol{y} = (y_1, \ldots, y_t)$. Let b_i bound the degree of $\det(P)$ in y_i (this determines the Kronecker substitution base, which must avoid overflow), and let $e_i \leq b_i$ bound the degree of a single entry of P in y_i. Define

$$\delta = e_t \prod_{i=1}^{t-1} (b_i + 1), \qquad N = \prod_{i=1}^{t} (b_i + 1),$$

where δ is the entry-wise univariate degree after Kronecker substitution. We also write $B := \max_i b_i$, L for the cost of one determinant probe, S for a sparsity bound on $\det(P)$, U for an upper bound on the support size of intermediate minors arising during a chosen evaluation strategy, μ for a dense monomial-count bound on one entry of P, and $\widetilde{M}(u)$ for soft-Oh multiplication at degree u.

Intermediate-minor support proxy. Several models (expansion, Bareiss) involve intermediate minors of P. Although a $j \times j$ minor is in general not a subpolynomial of $\det(P)$, for systems satisfying our genericity assumption (Definition 5) every entry of P has full support and no coefficient cancellation occurs in the determinantal expansion, so each intermediate minor's support is contained in the Minkowski sum of its row supports, which in turn is contained in $\mathrm{Supp}(\det P)$. Hence in the generic regime considered in this paper one may take $U \leq S$ throughout, which is the convention used in our complexity estimates.

I.1 Expansion

A baseline model is expansion by minors [34,43]. In the notation used in the code and plots, this is summarized by

$$\mathcal{T}^{\text{minor}} = \widetilde{O}(s 2^s \mu U).$$

The corresponding storage proxy is $\widetilde{O}(\binom{s}{s/2} U)$, since one keeps a full layer of minors rather than only the final determinant. This model is only competitive for very small matrices, but it is useful as a sanity check and as a lower-overhead baseline in the plots.

I.2 Bareiss

The fraction-free Bareiss elimination method [10] replaces cofactors by exact divisions inside a Gaussian-elimination style process. For symbolic polynomial entries, we use the coarse dense surrogate

$$\mathcal{T}^{\text{Bareiss}} = \widetilde{O}(s^3 U^2).$$

This captures the fact that the arithmetic count is cubic in the matrix size, while the dominant symbolic cost comes from multiplication/division on intermediate polynomials whose support is controlled, under genericity, by the structural support proxy U. The storage proxy is correspondingly $\widetilde{O}(s^2 U)$.

I.3 Kronecker

Kronecker substitution $y_i \mapsto t^{c_i}$ with $c_i = \prod_{j<i}(b_j + 1)$ maps the multivariate determinant problem to a univariate one. The substitution base $b_j + 1$ must be at least $\deg_{y_j} \det(P) + 1$ to avoid monomial collisions in the encoded determinant; Li [56] proves that under this condition the encoding is faithful, i.e. $\det(\Psi(P)) = \Psi(\det(P))$ (Theorems 5 and 7). After substitution each *entry* of P has t-degree at most δ above, so applying the HNF algorithm of [52] to the resulting univariate polynomial matrix gives

$$\mathcal{T}^{\text{HNF}} = \widetilde{O}(s^\omega \delta).$$

For the univariate final determinant ($k = 1$), this becomes the standard bound $\widetilde{O}(M^\omega E_4)$.

Correctness of Kronecker+HNF. Li [56] proves that Kronecker substitution commutes with the determinant (Theorems 5 and 7): if the substitution base c_i is chosen so that no monomial collisions can occur at the level of $\det(P)$, then

$$\Psi(\det(P)) = \det(\Psi(P)) \quad \text{and} \quad \det(P) = \Psi^{-1}\big(\det(\Psi(P))\big),$$

where Ψ denotes the Kronecker encoding and Ψ^{-1} its inverse. Since $\det(\Psi(P))$ is a uniquely defined univariate polynomial, *any* correct univariate determinant algorithm—including HNF [52]—produces the same output, regardless of which internal subroutines (kernel, column bases, etc.) it employs. We apply Ψ^{-1} only to the *final* univariate output, so the intermediate operations of HNF are immaterial to correctness. We have validated this experimentally on a wide range of mixed-degree systems: the Kronecker+HNF pipeline always produced the same determinant as expansion, Bareiss, and evaluation-interpolation.

I.4 Interpolation

Dense evaluation-interpolation [43] evaluates the determinant on the full tensor grid of size N and then reconstructs the polynomial variable by variable. The resulting model is

$$\mathcal{T}^{\text{eval}} = \widetilde{O}\left(NL + N\sum_{i=1}^{t} \widetilde{M}(b_i)\right) \subseteq \widetilde{O}(N(L + tB)).$$

The first term is the probe phase; the second is the tensor/Lagrange reconstruction phase. In the main text we use the simpler upper bound $\widetilde{O}(N(L+tB))$. This method is deterministic but becomes impractical as soon as the product grid N grows too quickly.

I.5 Sparse

When the determinant is sparse, derivative-based sparse interpolation over finite fields can be significantly better. We use the soft-Oh model inspired by Huang [44]:

$$\mathcal{T}^{\text{sparse}} = \widetilde{O}(LS + S\log q).$$

This is the *field-operation* count (intermediate step of [44, Theorem 12], before the final conversion to bit complexity); we use it because Steps 1–4 of the Dixon pipeline are uniformly counted in field operations over $\mathbb{F}_q$. The algorithm requires $\mathrm{char}(\mathbb{F}_q) \geq D$, where D is a partial-degree bound on the polynomial to be interpolated (Theorem 1 of [44]); this precondition is satisfied by the large-prime fields used in Poseidon and `XHash12` but *fails* for $\mathbb{F}_{2^\ell}$ as used by `Vision`, so the Sparse model is omitted from the `Vision` analysis. The $\log q$ factor is inherent to the algorithm: the term-locator polynomial of degree $\leq S$ is recovered by equal-degree factorization, which is a Las Vegas procedure of expected cost $\widetilde{O}(S\log q)$ field operations over $\mathbb{F}_q$ (Huang [44], Theorem 12). Unlike the $\log q$ factor that appears when converting field operations to bit operations, this one is already present in the field-operation count and is not negligible for the large prime or extension fields typical of AO primitives (e.g. $q \approx 2^{64}$). The crucial input is the term bound S. In the first-determinant model we use

$$S_1 \leq M^2 \binom{B_1 + k}{k},$$

where M is the Dixon-matrix size and $B_1 = nd$ bounds the total parameter degree. This bound is structural: the bilinear Dixon representation $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum X_u M_{u,v} \bar{X}_v$ has at most M^2 coefficient positions (the supports in $\boldsymbol{x}$ and in $\bar{\boldsymbol{x}}$ are symmetric, each of size at most M by Lemma 4), and each coefficient is a parameter polynomial of total degree at most B_1, hence carries at most $\binom{B_1+k}{k}$ monomials. For the final determinant, a dense monomial upper bound gives

$$S_4 \leq \binom{D_4 + k}{k}.$$

This is the key reason why sparse interpolation is the only first-determinant model that consistently stays below the final determinant in the high-variable/low-degree regime.

Stage-specific instantiations. For the first determinant we substitute $s = n$, $(\delta, N, S, L) = (\delta_1, N_1, S_1, L_1)$, and use $U = S_1$ as the intermediate-support proxy. For the final determinant we substitute $s = M$, $(\delta, N, S, L) = (\delta_4, N_4, S_4, L_4)$, and use $U = S_4$. This is exactly the convention used in the comparison plots and in the complexity-reporting code.

Where the parameters come from. The Kronecker bound δ_1 is read from the cancellation-matrix structure. In the uniform-degree case, a row-by-row analysis of the refined matrix $\widehat{C}$ gives

$$\deg_{x_i}(\det \widehat{C}) \le (i+1)d - 1, \qquad \deg_{\bar{x}_i}(\det \widehat{C}) \le (n-i)d - 1,$$

and the parameter variables satisfy $\deg_{a_j} \le d$ per row, so the total parameter degree of the $n \times n$ determinant is at most $B_1 = nd$. (See Appendix H for the derivation of these bounds.) These are exactly the ingredients used to form δ_1 and N_1 in the code. The sparse term bound S_1 comes from the Dixon-matrix construction itself: M^2 reflects the bilinear support size, while $\binom{B_1+k}{k}$ bounds the parameter contribution at each coefficient position.

For the final determinant, the HNF parameter δ_4 is obtained from the entry-wise parameter degree bound E_4, while N_4 and S_4 come from the total-degree bound D_4 of the elimination polynomial. In the well-determined case $k = 1$, this collapses to the univariate HNF model used in the main text.

Recursive block construction. Besides the five determinant models in Table 1, the code also keeps a Zhao/Fu-style recursive block-construction surrogate [76,62] that builds the Dixon matrix directly, bypassing the $n \times n$ cancellation determinant of Step 1:

$$T_2^{\mathrm{FDixon}} = \widetilde{O}\left(m_1^2 \, (n!)^3 \prod_{i=2}^{n} m_i^3 \right).$$

We do not list it in the main table because it is not a generic determinant model; rather, it is a special construction path that replaces the combined Step 1/Step 2 cost and can be attractive when the number of eliminated variables is small and the degrees are high.

J Poseidon

Poseidon [38] is a family of hash functions designed for efficient evaluation in zero-knowledge proof systems, while Poseidon2 [39] updates the linear layer for improved performance.

Let $p > 2^{30}$ be a prime and let $2 \le t \le 24$. Both Poseidon and Poseidon2 follow the HADES strategy. The permutation over $\mathbb{F}_p^t$ is

$$\mathcal{P}(x) = (\mathcal{E}_{R_F-1} \circ \cdots \circ \mathcal{E}_{R_F/2}) \circ (\mathcal{I}_{R_P-1} \circ \cdots \circ \mathcal{I}_0) \circ (\mathcal{E}_{R_F/2-1} \circ \cdots \circ \mathcal{E}_0) \circ M'(x),$$

where $\mathcal{E}$ denotes full rounds, $\mathcal{I}$ denotes partial rounds, and $R_F = 2R_f$. For Poseidon, M' is the identity.

The round functions are

$$\mathcal{E}_i(x) = M_E \cdot \big((x_0 + c_0^{(i)})^\alpha, \ldots, (x_{t-1} + c_{t-1}^{(i)})^\alpha\big),$$
$$\mathcal{I}_i(x) = M_I \cdot \big((x_0 + \bar{c}_0^{(i)})^\alpha, x_1, \ldots, x_{t-1}\big).$$

The parameters are chosen so that $\gcd(\alpha, p-1) = 1$ and so as to meet the targeted security level.

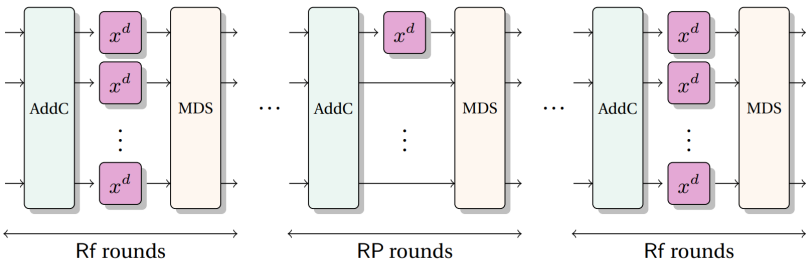

Fig. 9: Overview of Poseidon and Poseidon2. (Figure from [21])

Attack models used in Section 5. We follow the algebraic models from [40]. The direct Dixon method is applied to the input/output model, while the mixed-degree subspace models are estimated using the determinant bound from Corollary 1(3). This is the reason the Poseidon part of the main text only needs the final complexity plots and timing table.

K Vision

`Vision` is a ZK-friendly hash function following the *Marvellous* design strategy proposed in [4]. Over $\mathbb{F}_{2^\ell}$, each round alternates inverse maps with affine layers. For s branches, the round functions are

$$\mathcal{F}_{2i}(x) = M \cdot \left(B^{-1}(x_0^{-1}), \ldots, B^{-1}(x_{s-1}^{-1})\right) + C_{2i},$$
$$\mathcal{F}_{2i+1}(x) = M \cdot \left(B(x_0^{-1}), \ldots, B(x_{s-1}^{-1})\right) + C_{2i+1},$$

where $B(x) = b_0 + b_1 x + b_2 x^2 + b_3 x^4$ is an $\mathbb{F}_2$-affine linearized polynomial.

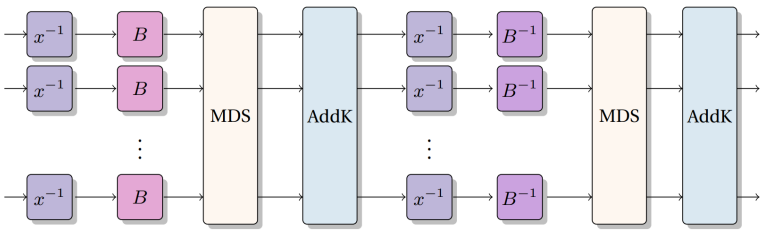

Fig. 10: Round function of `Vision`. (Figure from [21])

MITM model used in Section 5. For the CICO problem with $s = 2$, we follow the general MITM philosophy of [57] but use Dixon elimination instead

of an overdefined-system Gröbner-basis attack. The detailed intermediate variables and equations are listed below because they are longer than the high-level discussion needed in the main text.

For r rounds, we introduce intermediate variables $(w_{i,1}, w_{i,2})$ and $(z_{i,1}, z_{i,2})$ at round i $(2 \le i \le r)$. The core equations are:

$$F_{i,k}^{(1)} := B_3(w_{i,k}) \cdot \left(\sum_{j=1}^{s} M[k][j] \cdot B_3(z_{i-1,j}) + \epsilon_{i-1,k} \right) - 1, \qquad k \in \{1,2\}, \quad (38)$$

for the first inverse layer, and

$$F_{i,k}^{(2)} := z_{i,k} \cdot \left(\sum_{j=1}^{s} M[k][j] \cdot w_{i,j} + \sigma_{i,k} \right) - 1, \qquad k \in \{1,2\}, \quad (39)$$

for the second inverse layer.

The fixed-input and fixed-output constraints are represented by

$$f^{(\text{in})} := z_{1,1} \cdot \left(\sum_{j=1}^{s} M[1][j] \cdot w_{1,j} + \sigma_{1,1} + \beta_{3,1} \right) - 1, \quad (40)$$

and

$$f^{(\text{out})} := \sum_{j=1}^{s} M[0][j] \cdot B_3(z_{r,j}) + \epsilon_{r,0}. \quad (41)$$

Altogether this gives $4r - 2$ equations in $4r - 2$ variables (the $4r$ intermediate variables $w_{i,k}, z_{i,k}$ minus the two values fixed by the CICO constraints).

The equation-degree distribution used in the MITM estimate is summarized in Table 9.

Table 9: Degree distribution of equations for MITM elimination of `Vision`. Rows split each round count into the forward and backward chains; the three middle columns give the number of equations of degree 8, 2, and 4 in that chain; the last column reports the total degree of the resultant produced by that chain.

r	Equation	# Equations			Degree
		deg = 8	deg = 2	deg = 4	
even	Forward	r	r	0	2^{4r}
	Backward	$r-2$	r	1	2^{4r-4}
odd	Forward	$r-1$	$r+1$	0	2^{4r-2}
	Backward	$r-1$	$r-1$	1	2^{4r-2}

The MITM elimination proceeds by computing forward resultants up to the middle state and backward resultants from the output side to the same middle

state. The two shared middle variables $\mathcal{Z}$ are retained throughout, so the forward chain terminates in one relation in $\mathcal{Z}$ of total degree D_{fwd} and the backward chain in another of total degree D_{bwd}. The final step is a single resultant within $\mathcal{Z}$: it eliminates one of the two middle variables between these two relations, yielding the univariate eliminant in the remaining middle variable. This is the detailed model underlying Equation (24) in the main text.

L XHash12

`XHash12` is a STARK-friendly sponge hash function proposed by Ashur et al. [5]. The permutation over $\mathbb{F}_p^{12}$ is

$$\mathcal{F}(x) = M \circ \mathcal{B}_2 \circ \mathcal{I}_2 \circ \mathcal{P}_2 \circ \mathcal{B}_1 \circ \mathcal{I}_1 \circ \mathcal{P}_1 \circ \mathcal{B}_0 \circ \mathcal{I}_0 \circ \mathcal{P}_0(x),$$

where $p = 2^{64} - 2^{32} + 1$. The nonlinear layers combine power maps over $\mathbb{F}_p$ and over $\mathbb{F}_{p^3}$.

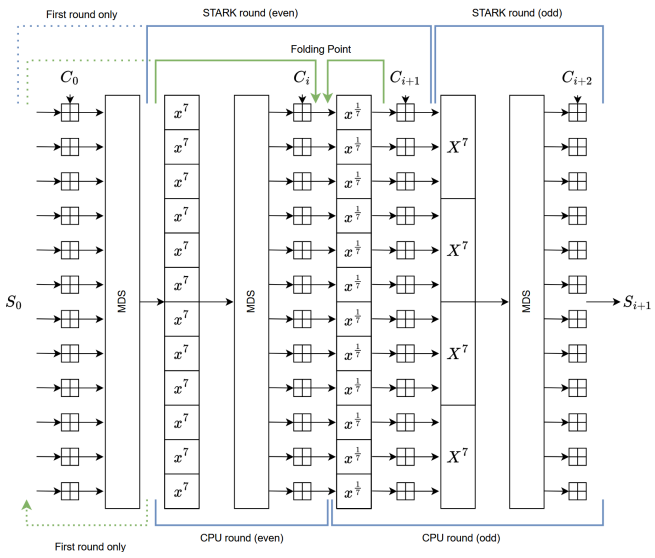

Fig. 11: Round function of `XHash12` [5]

Hybrid model used in Section 5. For an s-step instance, let $n = 12s$ and write the system in triangular form

$$\mathcal{P}_k = \begin{cases} Z_1^{\alpha} - f_1(\mathcal{X}) = 0 \\ \vdots \\ Z_n^{\alpha} - f_n(\mathcal{X}, Z_1, \ldots, Z_{n-1}) = 0 \\ h_1(\mathcal{X}, Z_1, \ldots, Z_n) = 0 \\ \vdots \\ h_k(\mathcal{X}, Z_1, \ldots, Z_n) = 0 \end{cases}$$

with $\alpha = 7$ for the standard XHash12 instance.

For CICO-1, the reduction method of [11] can be applied directly. For larger k, the hybrid strategy first derives reduced relations $R_1(\mathcal{X}), \ldots, R_k(\mathcal{X})$ from the triangular ideal and then performs a final Dixon elimination on these relations. The main text reports only the resulting complexities and timings because the qualitative conclusion is simple: the hybrid strategy is excellent for very small k, but the direct Dixon method remains the relevant estimate for the natural CICO-4 setting.